

KAN Research Preparation Notes

# Mathematical Foundations for Kolmogorov–Arnold Networks

A Continuous Study Record

J Song (Alex)

# Contents

|          |    |
|----------|----|
| Contents | ii |
|----------|----|

|                       |          |
|-----------------------|----------|
| <b>LINEAR ALGEBRA</b> | <b>1</b> |
|-----------------------|----------|

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>1 Vectors, Matrices, and Column Space</b>                              | <b>2</b>  |
| 1.1 The matrix form of a linear system . . . . .                          | 2         |
| 1.2 The row picture: equations as constraints . . . . .                   | 2         |
| 1.3 The column picture: targets as combinations . . . . .                 | 3         |
| 1.3.1 When is every target reachable? . . . . .                           | 3         |
| 1.4 Matrix–vector multiplication, two readings . . . . .                  | 4         |
| 1.5 Span, independence, and the shape of a solution set . . . . .         | 5         |
| 1.6 Notation used in this book . . . . .                                  | 5         |
| <b>2 Solving <math>Ax = b</math>: Elimination and <math>A = LU</math></b> | <b>7</b>  |
| 2.1 Elimination on an example . . . . .                                   | 7         |
| 2.2 Back-substitution . . . . .                                           | 7         |
| 2.3 Elimination matrices . . . . .                                        | 8         |
| 2.4 Matrix multiplication, four ways . . . . .                            | 8         |
| 2.5 Row exchanges and permutation matrices . . . . .                      | 9         |
| 2.6 When elimination fails . . . . .                                      | 10        |
| 2.7 Inverses and Gauss–Jordan . . . . .                                   | 10        |
| 2.8 The factorisation $A = LU$ . . . . .                                  | 11        |
| 2.9 Transposes and symmetry . . . . .                                     | 12        |
| 2.10 The cost of elimination . . . . .                                    | 12        |
| <b>3 The Four Fundamental Subspaces</b>                                   | <b>13</b> |
| 3.1 Vector spaces and subspaces . . . . .                                 | 13        |
| 3.2 The column space . . . . .                                            | 13        |
| 3.3 The nullspace . . . . .                                               | 13        |
| 3.4 Rank, echelon form, and $A = CR$ . . . . .                            | 14        |
| 3.5 The complete solution of $Ax = b$ . . . . .                           | 15        |
| 3.6 Independence, basis, and dimension . . . . .                          | 15        |
| 3.7 The four subspaces . . . . .                                          | 15        |
| <b>4 Orthogonality, Projection, and Least Squares</b>                     | <b>17</b> |
| 4.1 Inner products, length, and angle . . . . .                           | 17        |
| 4.2 Orthogonal subspaces . . . . .                                        | 17        |
| 4.3 Projection onto a line . . . . .                                      | 18        |
| 4.4 Projection onto a subspace . . . . .                                  | 18        |
| 4.5 Least squares . . . . .                                               | 18        |
| 4.6 Orthonormal bases and Gram–Schmidt . . . . .                          | 19        |
| 4.7 The factorisation $A = QR$ . . . . .                                  | 19        |
| <b>5 Determinants</b>                                                     | <b>21</b> |
| 5.1 Three properties that define it . . . . .                             | 21        |
| 5.2 What follows immediately . . . . .                                    | 21        |
| 5.3 Determinant from the pivots . . . . .                                 | 22        |
| 5.4 Cofactors and the big formula . . . . .                               | 22        |
| 5.5 Three uses . . . . .                                                  | 23        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>6</b>  | <b>Eigenvalues and Eigenvectors</b>                      | <b>24</b> |
| 6.1       | The defining equation . . . . .                          | 24        |
| 6.2       | Trace and determinant . . . . .                          | 24        |
| 6.3       | Diagonalisation and powers . . . . .                     | 25        |
| 6.4       | Symmetric matrices . . . . .                             | 25        |
| 6.5       | Positive definite matrices . . . . .                     | 26        |
| 6.6       | Similar matrices . . . . .                               | 26        |
| <b>7</b>  | <b>The Singular Value Decomposition</b>                  | <b>27</b> |
| 7.1       | The statement . . . . .                                  | 27        |
| 7.2       | Where it comes from . . . . .                            | 27        |
| 7.3       | The four subspaces, one picture . . . . .                | 28        |
| 7.4       | Rank-one pieces and best approximation . . . . .         | 28        |
| 7.5       | The pseudoinverse . . . . .                              | 28        |
| <b>8</b>  | <b>Linear Transformations and Change of Basis</b>        | <b>29</b> |
| 8.1       | What makes a map linear . . . . .                        | 29        |
| 8.2       | The matrix of a transformation . . . . .                 | 29        |
| 8.3       | Change of basis . . . . .                                | 30        |
| 8.4       | Choosing the basis well . . . . .                        | 30        |
| <b>9</b>  | <b>Linear Algebra in Optimisation</b>                    | <b>31</b> |
| 9.1       | Norms . . . . .                                          | 31        |
| 9.2       | Conditioning . . . . .                                   | 31        |
| 9.3       | Quadratic forms . . . . .                                | 31        |
| 9.4       | Regularisation . . . . .                                 | 32        |
| <b>10</b> | <b>Linear Algebra behind Learning from Data</b>          | <b>33</b> |
| 10.1      | Data as a matrix . . . . .                               | 33        |
| 10.2      | Centring and covariance . . . . .                        | 33        |
| 10.3      | Principal components . . . . .                           | 34        |
| 10.4      | Low rank as a modelling assumption . . . . .             | 34        |
| 10.5      | Batched computation . . . . .                            | 34        |
| 10.6      | What carries forward . . . . .                           | 35        |
|           | <b>CALCULUS FOR LEARNING</b>                             | <b>36</b> |
| <b>11</b> | <b>Derivatives and the Chain Rule</b>                    | <b>37</b> |
| 11.1      | The derivative as a local linear approximation . . . . . | 37        |
| 11.2      | The rules . . . . .                                      | 37        |
| 11.3      | The chain rule . . . . .                                 | 38        |
| 11.4      | Long compositions . . . . .                              | 38        |
| 11.5      | What is not covered . . . . .                            | 38        |
| <b>12</b> | <b>Partial Derivatives and Gradients</b>                 | <b>39</b> |
| 12.1      | Partial derivatives . . . . .                            | 39        |
| 12.2      | Direction of steepest ascent . . . . .                   | 39        |
| 12.3      | Gradients of the expressions that matter . . . . .       | 40        |
| 12.4      | A worked parameter gradient . . . . .                    | 40        |
| 12.5      | Convexity in one line . . . . .                          | 40        |
| <b>13</b> | <b>Jacobians, Hessians, and Taylor Approximation</b>     | <b>42</b> |
| 13.1      | The Jacobian . . . . .                                   | 42        |
| 13.2      | The chain rule in matrix form . . . . .                  | 42        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| 13.3      | The Hessian . . . . .                                    | 43        |
| 13.4      | Taylor approximation . . . . .                           | 43        |
| 13.5      | The Jacobian determinant . . . . .                       | 43        |
| <b>14</b> | <b>Gradient-Based Optimisation</b>                       | <b>44</b> |
| 14.1      | Descent directions and the basic method . . . . .        | 44        |
| 14.2      | The quadratic case, exactly . . . . .                    | 44        |
| 14.3      | Momentum and adaptive steps . . . . .                    | 45        |
| 14.4      | Newton's method . . . . .                                | 45        |
| 14.5      | Stochastic gradients . . . . .                           | 45        |
| 14.6      | Backpropagation . . . . .                                | 45        |
|           | <b>PROBABILITY FOR LEARNING</b>                          | <b>47</b> |
| <b>15</b> | <b>Sample Spaces, Conditioning, and Bayes</b>            | <b>48</b> |
| 15.1      | Sample spaces and the axioms . . . . .                   | 48        |
| 15.2      | Conditional probability . . . . .                        | 48        |
| 15.3      | Total probability and Bayes' rule . . . . .              | 48        |
| 15.4      | Independence . . . . .                                   | 49        |
| <b>16</b> | <b>Random Variables and Distributions</b>                | <b>50</b> |
| 16.1      | Discrete random variables . . . . .                      | 50        |
| 16.2      | Continuous random variables . . . . .                    | 50        |
| 16.3      | Joint, marginal, and conditional distributions . . . . . | 51        |
| 16.4      | Transforming a random variable . . . . .                 | 51        |
| <b>17</b> | <b>Expectation, Variance, and Covariance</b>             | <b>52</b> |
| 17.1      | Expectation . . . . .                                    | 52        |
| 17.2      | Expectations of functions . . . . .                      | 52        |
| 17.3      | Variance . . . . .                                       | 52        |
| 17.4      | Covariance and correlation . . . . .                     | 53        |
| 17.5      | The covariance matrix . . . . .                          | 53        |
| 17.6      | Conditional expectation . . . . .                        | 54        |
| <b>18</b> | <b>Limit Theorems</b>                                    | <b>55</b> |
| 18.1      | Two inequalities . . . . .                               | 55        |
| 18.2      | The law of large numbers . . . . .                       | 55        |
| 18.3      | The central limit theorem . . . . .                      | 56        |
| 18.4      | What the theorems do not say . . . . .                   | 56        |
| <b>19</b> | <b>Estimation</b>                                        | <b>57</b> |
| 19.1      | The problem . . . . .                                    | 57        |
| 19.2      | Maximum likelihood . . . . .                             | 57        |
| 19.3      | Two losses, one principle . . . . .                      | 58        |
| 19.4      | Bayesian estimation . . . . .                            | 58        |
| 19.5      | What this part establishes . . . . .                     | 59        |
|           | <b>DEEP LEARNING FOUNDATIONS</b>                         | <b>60</b> |
| <b>20</b> | <b>What a Learning Problem Is</b>                        | <b>61</b> |
| 20.1      | Vocabulary . . . . .                                     | 61        |
| 20.2      | The four components . . . . .                            | 61        |
| 20.2.1    | Data . . . . .                                           | 61        |

|           |                                                                 |               |
|-----------|-----------------------------------------------------------------|---------------|
| 20.2.2    | Model . . . . .                                                 | 62            |
| 20.2.3    | Objective function . . . . .                                    | 62            |
| 20.2.4    | Optimisation algorithm . . . . .                                | 62            |
| 20.3      | Training, testing, and overfitting . . . . .                    | 62            |
| 20.4      | Why the framing is worth keeping . . . . .                      | 63            |
| <b>21</b> | <b>A Map of Learning Problems</b>                               | <b>64</b>     |
| 21.1      | Supervised learning . . . . .                                   | 64            |
| 21.1.1    | Regression . . . . .                                            | 64            |
| 21.1.2    | Classification . . . . .                                        | 64            |
| 21.1.3    | Search and ranking . . . . .                                    | 64            |
| 21.1.4    | Recommender systems . . . . .                                   | 65            |
| 21.1.5    | Sequence learning . . . . .                                     | 65            |
| 21.2      | Unsupervised learning . . . . .                                 | 65            |
| 21.3      | Interacting with an environment . . . . .                       | 65            |
| 21.4      | Reinforcement learning . . . . .                                | 66            |
| 21.5      | Terminology carried forward . . . . .                           | 66            |
|           | <br><b>WHERE THE HALVES MEET</b>                                | <br><b>67</b> |
| <b>22</b> | <b>From Column Space to Linear Layers</b>                       | <b>68</b>     |
| 22.1      | A network is a composition of the objects already met . . . . . | 68            |
| 22.2      | Three questions, three answers already given . . . . .          | 68            |
| 22.3      | Where the linear picture runs out . . . . .                     | 69            |
| 22.4      | What to read next . . . . .                                     | 69            |

# LINEAR ALGEBRA

# 1 Vectors, Matrices, and Column Space

|       |                                                     |   |
|-------|-----------------------------------------------------|---|
| 1.1   | The matrix form of a linear system . . . . .        | 2 |
| 1.2   | The row picture: equations as constraints . . . . . | 2 |
| 1.3   | The column picture: targets as combinations . . .   | 3 |
| 1.3.1 | When is every target reachable? . . . . .           | 3 |
| 1.4   | Matrix–vector multiplication, two readings . . . .  | 4 |
| 1.5   | Span, independence, and the shape of a solution set | 5 |
| 1.6   | Notation used in this book                          | 5 |

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], §1.1–1.4 and §2.1; MIT 18.06 [7], Lecture 1.

Linear algebra begins with a bookkeeping question — how to store a system of equations — and almost immediately turns into a geometric one. This chapter sets up the notation used for the rest of the book and introduces the two readings of  $Ax = b$  that everything later depends on.

## 1.1 The matrix form of a linear system

A system of  $m$  linear equations in  $n$  unknowns can be packed into a single product,

$$Ax = b,$$

where  $A \in \mathbb{R}^{m \times n}$  holds the coefficients,  $x \in \mathbb{R}^n$  holds the unknowns, and  $b \in \mathbb{R}^m$  is the target. Throughout this section I work with

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -3 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ -1 \\ 4 \end{bmatrix}.$$

Read one row at a time, the product unpacks into the three equations

$$\begin{aligned} 2x - y &= 0, \\ -x + 2y - z &= -1, \\ -3y + 4z &= 4. \end{aligned}$$

Nothing has been solved yet. The matrix form is a change of notation, not a change of content — but the notation is what makes the two pictures below visible.

Substituting  $x = [0 \ 0 \ 1]^T$  satisfies all three equations at once, so this vector solves the system. The interesting question is not *whether* this particular vector works, but how one would have found it, and what the two ways of reading the product suggest.

## 1.2 The row picture: equations as constraints

Each row of a three-variable system describes a plane in  $\mathbb{R}^3$ . A vector solves the system exactly when it lies on all three planes simultaneously, so the solution set is the intersection of the three planes. For the example above that intersection is the single point  $(0, 0, 1)$ .

**Remark 1.2.1** The row picture is a picture of *constraints*. Every additional equation removes a degree of freedom, and a solution is whatever survives all of the restrictions at once.

Its usefulness is bounded by the fact that it has to be drawn. With nine unknowns the same reasoning would require intersecting hyperplanes in  $\mathbb{R}^9$ , which no amount of care will make visible. The algebra is unaffected — elimination will still run — but the geometric intuition stops paying for itself. That is the first reason to look for a second reading of the same product.

### 1.3 The column picture: targets as combinations

Write  $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$  for the columns of  $A$ . Expanding the product by columns instead of by rows gives

$$A\mathbf{x} = x\mathbf{a}_1 + y\mathbf{a}_2 + z\mathbf{a}_3,$$

so the system asks for coefficients that mix the three columns into the target:

$$x \begin{bmatrix} 2 \\ -1 \\ 0 \end{bmatrix} + y \begin{bmatrix} -1 \\ 2 \\ -3 \end{bmatrix} + z \begin{bmatrix} 0 \\ -1 \\ 4 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 4 \end{bmatrix}.$$

In this instance the answer can be read off without any calculation: the target is the third column, so  $(x, y, z) = (0, 0, 1)$ . The example is convenient, but the idea it illustrates is not.

**Definition 1.3.1** A vector  $\mathbf{b}$  is a linear combination of  $\mathbf{a}_1, \dots, \mathbf{a}_n$  when there exist scalars  $x_1, \dots, x_n$  with

$$\mathbf{b} = x_1\mathbf{a}_1 + \dots + x_n\mathbf{a}_n.$$

The set of all such combinations is the column space of  $A$ , written  $\text{Col}(A)$ .

The definition converts a question about solving into a question about membership:

$$A\mathbf{x} = \mathbf{b} \text{ is solvable} \iff \mathbf{b} \in \text{Col}(A).$$

Changing the target tests a different recipe built from the same ingredients. Taking one of the first column and one of the second,

$$\begin{bmatrix} 2 \\ -1 \\ 0 \end{bmatrix} + \begin{bmatrix} -1 \\ 2 \\ -3 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ -3 \end{bmatrix},$$

so the system with target  $\begin{bmatrix} 1 & 1 & -3 \end{bmatrix}^T$  is solved by  $\mathbf{x} = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}^T$ .

The row picture asks whether a candidate satisfies every restriction. The column picture asks which recipe of the available ingredients produces the target. Both are correct; they suggest different algorithms.

#### 1.3.1 When is every target reachable?

For a square matrix  $A \in \mathbb{R}^{n \times n}$  the natural question is whether  $A\mathbf{x} = \mathbf{b}$  can be solved for every  $\mathbf{b} \in \mathbb{R}^n$ . In the language just introduced this asks whether the columns fill the whole space,

$$\text{Col}(A) = \mathbb{R}^n.$$



If the columns are linearly independent they span  $\mathbb{R}^n$ , every target is reachable, and the coefficients are unique. The matrix is then invertible and  $\mathbf{x} = A^{-1}\mathbf{b}$ .

If instead the columns are dependent, at least one of them contributes no new direction: it already lies in the span of the others. In  $\mathbb{R}^3$  the extreme case is three columns lying in a common plane. Every combination stays in that plane, so any target off the plane is unreachable, while targets on it are reached in infinitely many ways.

**Remark 1.3.1** Dependence and invertibility are the same phenomenon seen from two sides. Independent columns give exactly one solution for each of the  $n$ -dimensional family of targets; dependent columns give either none or infinitely many, depending on where the target sits.

This dichotomy — invertible against singular — is the thread that runs through elimination in Chapter 2, the four subspaces in Chapter 3, and the singular value decomposition in Chapter 7. It reappears, transformed, whenever a network layer is asked whether it can represent a given output.

## 1.4 Matrix–vector multiplication, two readings

The column viewpoint supplies the definition of the product that is easiest to reason with. If

$$A = \begin{bmatrix} | & & | \\ \mathbf{a}_1 & \cdots & \mathbf{a}_n \\ | & & | \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix},$$

then

$$A\mathbf{x} = x_1\mathbf{a}_1 + \cdots + x_n\mathbf{a}_n. \quad (1.1)$$

The rule taught first in school computes one entry at a time: entry  $i$  of  $A\mathbf{x}$  is the dot product of row  $i$  of  $A$  with  $\mathbf{x}$ . Both give the same numbers. Taking

$$A = \begin{bmatrix} 2 & 5 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix},$$

the column reading gives

$$1 \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 2 \begin{bmatrix} 5 \\ 3 \end{bmatrix} = \begin{bmatrix} 12 \\ 7 \end{bmatrix},$$

and the row reading gives the same vector through  $2 \cdot 1 + 5 \cdot 2 = 12$  and  $1 \cdot 1 + 3 \cdot 2 = 7$ .

**Remark 1.4.1** The two readings are not rivals; they answer different questions. Row by row is how the product is *computed* entry by entry. Column by column is what the product *means*:  $A\mathbf{x}$  never leaves the column space of  $A$ , no matter which  $\mathbf{x}$  is supplied.

That last observation is worth stating on its own, because it is the reason Equation 1.1 is the form used throughout this book.

**Proposition 1.4.1** *For every  $\mathbf{x} \in \mathbb{R}^n$  the product  $A\mathbf{x}$  lies in  $\text{Col}(A)$ , and every element of  $\text{Col}(A)$  arises as  $A\mathbf{x}$  for some  $\mathbf{x}$ .*

*Proof.* Equation 1.1 exhibits  $A\mathbf{x}$  as a linear combination of the columns, which is membership in  $\text{Col}(A)$  by definition. Conversely a combination  $x_1\mathbf{a}_1 + \dots + x_n\mathbf{a}_n$  is the product of  $A$  with the coefficient vector  $\mathbf{x} = [x_1, \dots, x_n]^\top$ .  $\square$

## 1.5 Span, independence, and the shape of a solution set

Two definitions make the earlier discussion precise.

**Definition 1.5.1** *The span of  $\mathbf{a}_1, \dots, \mathbf{a}_n$  is the set of all their linear combinations. The vectors are linearly independent when*

$$x_1\mathbf{a}_1 + \dots + x_n\mathbf{a}_n = \mathbf{0} \quad \text{forces} \quad x_1 = \dots = x_n = 0,$$

*and linearly dependent otherwise.*

Dependence is exactly the statement that some non-zero  $\mathbf{x}$  satisfies  $A\mathbf{x} = \mathbf{0}$ . Suppose such an  $\mathbf{x}_0 \neq \mathbf{0}$  exists and  $\mathbf{x}_p$  solves  $A\mathbf{x} = \mathbf{b}$ . Then for every scalar  $c$ ,

$$A(\mathbf{x}_p + c\mathbf{x}_0) = A\mathbf{x}_p + cA\mathbf{x}_0 = \mathbf{b} + \mathbf{0} = \mathbf{b},$$

so one solution has multiplied into infinitely many. This is the whole explanation for the trichotomy that a linear system obeys.

**Theorem 1.5.1** *A system  $A\mathbf{x} = \mathbf{b}$  has no solution, exactly one solution, or infinitely many. It never has, for instance, exactly two.*

*Proof.* If there is no solution the first case holds. If  $\mathbf{x}_p$  and  $\mathbf{x}_q$  are distinct solutions then  $\mathbf{x}_0 = \mathbf{x}_p - \mathbf{x}_q$  is non-zero and satisfies  $A\mathbf{x}_0 = \mathbf{0}$ ; the displayed calculation above then produces a distinct solution for every scalar  $c$ , so there are infinitely many. Hence two or more solutions already forces infinitely many.  $\square$

The set of vectors killed by  $A$  therefore deserves a name of its own; it is the nullspace, and Chapter 3 is largely about it.

## 1.6 Notation used in this book

Lower-case bold letters denote column vectors, capitals denote matrices, and  $A^\top$  is the transpose. The columns of  $A$  are  $\mathbf{a}_1, \dots$ ; the entry in row  $i$  and column  $j$  is  $a_{ij}$ . The symbol  $\mathbb{R}^n$  is the set of real column vectors of length  $n$ , and  $I$  is the identity matrix whose size is always clear from context.

Vectors are columns unless stated otherwise, so a row vector is written  $\mathbf{v}^\top$ . Data matrices are the one place where this convention grates: a batch of  $n$  examples with  $d$  features is stored as  $X \in \mathbb{R}^{n \times d}$  with one example per

Independence is a statement about the zero target only, yet it controls the solution count for every target at once. That leverage is what makes it the central definition of the subject.

*row*, which is the convention every numerical library uses. Whenever both appear together the shapes are stated explicitly rather than left to be inferred.

# Solving $Ax = b$ : Elimination and $A = LU$

# 2

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], §1.4 and §2.2–2.7; MIT 18.06 [7], Lectures 2–5.

Chapter 1 explained what a solution is. This chapter is about how to find one without guessing, and about what the procedure leaves behind: a factorisation of  $A$  that is worth more than the solution it was invented to produce.

## 2.1 Elimination on an example

Take

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 8 & 1 \\ 0 & 4 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 2 \\ 12 \\ 2 \end{bmatrix},$$

and work with the augmented array  $[A \quad \mathbf{b}]$  so that the right-hand side receives every operation the left-hand side does.

The first *pivot* is the entry  $a_{11} = 1$ . Subtracting  $3 \times \text{row 1}$  from row 2 clears the entry below it, and the  $(3, 1)$  entry is already zero, so nothing is needed there:

$$\left[ \begin{array}{ccc|c} 1 & 2 & 1 & 2 \\ 3 & 8 & 1 & 12 \\ 0 & 4 & 1 & 2 \end{array} \right] \rightarrow \left[ \begin{array}{ccc|c} 1 & 2 & 1 & 2 \\ 0 & 2 & -2 & 6 \\ 0 & 4 & 1 & 2 \end{array} \right].$$

The second pivot is the 2 now sitting in position  $(2, 2)$ . Subtracting twice row 2 from row 3 clears below it:

$$\rightarrow \left[ \begin{array}{ccc|c} 1 & 2 & 1 & 2 \\ 0 & 2 & -2 & 6 \\ 0 & 0 & 5 & -10 \end{array} \right] = [U \mid \mathbf{c}].$$

The left block  $U$  is *upper triangular*, the pivots are 1, 2, 5, and the transformed right-hand side is  $\mathbf{c} = [2, 6, -10]^T$ .

**Definition 2.1.1** A pivot is the first non-zero entry in a row at the moment that row is used to clear the entries beneath it. The positions of the pivots, not their values, determine the structure of the solution set.

## 2.2 Back-substitution

Triangular systems are solved from the bottom up. Reading  $U\mathbf{x} = \mathbf{c}$  as equations,

$$\begin{aligned} x + 2y + z &= 2, \\ 2y - 2z &= 6, \\ 5z &= -10, \end{aligned}$$

|      |                                                  |    |
|------|--------------------------------------------------|----|
| 2.1  | Elimination on an example                        | 7  |
| 2.2  | Back-substitution . . . . .                      | 7  |
| 2.3  | Elimination matrices . . .                       | 8  |
| 2.4  | Matrix multiplication, four ways . . . . .       | 8  |
| 2.5  | Row exchanges and permutation matrices . . . . . | 9  |
| 2.6  | When elimination fails . .                       | 10 |
| 2.7  | Inverses and Gauss–Jordan                        | 10 |
| 2.8  | The factorisation $A = LU$ .                     | 11 |
| 2.9  | Transposes and symmetry                          | 12 |
| 2.10 | The cost of elimination . .                      | 12 |

The multiplier 3 is the number of row 1 subtracted from row 2. It is worth recording rather than discarding: Section 2.8 assembles all the multipliers into a matrix.

the last equation gives  $z = -2$ . Substituting into the second,  $2y - 2(-2) = 6$ , so  $2y = 2$  and  $y = 1$ . Substituting both into the first,  $x + 2(1) + (-2) = 2$ , so  $x = 2$ . Hence

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ -2 \end{bmatrix},$$

and substituting into the original  $A\mathbf{x}$  returns  $[2, 6 + 8 - 2, 4 - 2]^T = [2, 12, 2]^T = \mathbf{b}$ , as required.

Always substitute back into the *original* matrix, not into  $U$ . An arithmetic slip made during elimination is invisible to a check performed on the eliminated system.

## 2.3 Elimination matrices

Each elimination step is itself a matrix multiplication. Subtracting three copies of row 1 from row 2 is achieved by left multiplication with

$$E_{21} = \begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

which differs from the identity in exactly one off-diagonal entry. Indeed  $E_{21}A$  reproduces the first array computed above. The second step is

$$E_{32} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix}, \quad E_{32}(E_{21}A) = U.$$

Because matrix multiplication is associative, the two steps can be combined before touching  $A$ :

$$(E_{32}E_{21})A = U.$$

**Remark 2.3.1** Associativity is doing real work here. It is what allows a sequence of row operations to be replaced by a single matrix, and it is why the whole elimination process can be recorded as a factorisation instead of as a script of instructions.

Undoing a step is as easy as performing it: if  $E_{21}$  subtracts three copies of row 1 from row 2, then adding them back inverts it,

$$E_{21}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad E_{21}^{-1}E_{21} = I.$$

## 2.4 Matrix multiplication, four ways

Since elimination is multiplication, it is worth collecting the ways a product  $C = AB$  with  $A \in \mathbb{R}^{m \times n}$  and  $B \in \mathbb{R}^{n \times p}$  can be read. All four produce the same  $C$ ; each makes a different fact obvious.

**Entry by entry**  $c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}$ , the dot product of row  $i$  of  $A$  with column  $j$  of  $B$ .

**Column by column** column  $j$  of  $C$  is  $A\mathbf{b}_j$ . Hence every column of  $C$  is a combination of the columns of  $A$ , and  $\text{Col}(AB) \subseteq \text{Col}(A)$ .

**Row by row** row  $i$  of  $C$  is  $\mathbf{a}_i^\top B$ , a combination of the rows of  $B$ . Hence the row space of  $AB$  sits inside the row space of  $B$ .

**Column times row**  $C = \sum_{k=1}^n \mathbf{a}_k \mathbf{b}_k^\top$ , a sum of  $n$  rank-one matrices, where  $\mathbf{a}_k$  is column  $k$  of  $A$  and  $\mathbf{b}_k^\top$  is row  $k$  of  $B$ .

**Example 2.4.1** With

$$A = \begin{bmatrix} 2 & 7 \\ 3 & 8 \\ 4 & 9 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 6 \\ 0 & 0 \end{bmatrix},$$

the column-times-row reading gives

$$AB = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & 6 \end{bmatrix} + \begin{bmatrix} 7 \\ 8 \\ 9 \end{bmatrix} \begin{bmatrix} 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 12 \\ 3 & 18 \\ 4 & 24 \end{bmatrix}.$$

The second term vanishes because the second row of  $B$  is zero, which the entry-by-entry rule would have obscured behind six separate multiplications.

The fourth reading is the one that generalises furthest. The singular value decomposition in Chapter 7 is precisely a column-times-row expansion in which the terms are ordered by importance.

A fifth reading, block multiplication, follows from the same algebra: if  $A$  and  $B$  are cut into conforming blocks then

$$\begin{bmatrix} A_1 & A_2 \\ A_3 & A_4 \end{bmatrix} \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix} = \begin{bmatrix} A_1 B_1 + A_2 B_3 & A_1 B_2 + A_2 B_4 \\ A_3 B_1 + A_4 B_3 & A_3 B_2 + A_4 B_4 \end{bmatrix},$$

provided the block shapes match. Every practical implementation of matrix multiplication — and every batched computation in a neural network — relies on this.

## 2.5 Row exchanges and permutation matrices

A zero in the pivot position is not fatal on its own. If a row further down has a non-zero entry in that column, exchanging the two rows restores a usable pivot. The exchange is again a matrix: for two rows,

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad P \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} c & d \\ a & b \end{bmatrix}.$$

**Remark 2.5.1** Multiplying on the *left* rearranges rows; multiplying on the *right* rearranges columns. The same  $P$  gives

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} P = \begin{bmatrix} b & a \\ d & c \end{bmatrix}.$$

Side matters, and the habit of checking which side an operation acts on prevents a large fraction of avoidable errors.

**Definition 2.5.1** A permutation matrix is obtained from  $I$  by reordering its rows. Every permutation matrix satisfies  $P^{-1} = P^\top$ , because the reordering is undone by the reverse reordering.

With exchanges allowed, elimination on an invertible matrix always succeeds, and the factorisation of Section 2.8 becomes  $PA = LU$ .

## 2.6 When elimination fails

Failure occurs only when a pivot position holds zero *and* every entry below it in that column is zero as well: there is nothing left to exchange with. A schematic third stage of the form

$$\begin{bmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & 0 \end{bmatrix}$$

has only two pivots for three columns. The matrix is then singular, and by Subsection 1.3.1 some targets are unreachable while others are reached in infinitely many ways.

**Remark 2.6.1** It is worth separating two situations that look alike. A zero *encountered* in a pivot position is a bookkeeping accident, repaired by a row exchange. A zero that *survives* every available exchange is a statement about the matrix: its columns are dependent, and no algorithm can repair that.

## 2.7 Inverses and Gauss–Jordan

For a square matrix,  $A^{-1}$  is the matrix with  $A^{-1}A = I = AA^{-1}$ . It exists exactly when  $A$  is non-singular. The quickest certificate of *non*-existence is a non-zero vector in the nullspace.

**Proposition 2.7.1** *If  $A\mathbf{x} = \mathbf{0}$  for some  $\mathbf{x} \neq \mathbf{0}$ , then  $A$  has no inverse.*

*Proof.* Suppose  $A^{-1}$  existed. Multiplying  $A\mathbf{x} = \mathbf{0}$  on the left gives  $\mathbf{x} = A^{-1}\mathbf{0} = \mathbf{0}$ , contradicting  $\mathbf{x} \neq \mathbf{0}$ .  $\square$

**Example 2.7.1** For  $A = \begin{bmatrix} 1 & 3 \\ 2 & 6 \end{bmatrix}$  the vector  $\mathbf{x} = \begin{bmatrix} 3 & -1 \end{bmatrix}^T$  gives

$$\begin{bmatrix} 1 & 3 \\ 2 & 6 \end{bmatrix} \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

so this  $A$  is singular. The second column is three times the first, which is the same fact stated in the column picture.

To *find* an inverse, note that column  $j$  of  $A^{-1}$  is the solution of  $A\mathbf{x} = \mathbf{e}_j$ , where  $\mathbf{e}_j$  is column  $j$  of  $I$ . The  $n$  systems share the same left-hand side, so they can be eliminated together. Gauss–Jordan places  $A$  and  $I$  side by side and reduces until the left block is the identity:

$$[A \quad I] \longrightarrow [I \quad A^{-1}].$$

**Example 2.7.2** Take  $A = \begin{bmatrix} 1 & 3 \\ 2 & 7 \end{bmatrix}$ . Subtracting 2×row 1 from row 2, then 3×the new row 2 from row 1:

$$\left[ \begin{array}{cc|cc} 1 & 3 & 1 & 0 \\ 2 & 7 & 0 & 1 \end{array} \right] \rightarrow \left[ \begin{array}{cc|cc} 1 & 3 & 1 & 0 \\ 0 & 1 & -2 & 1 \end{array} \right] \rightarrow \left[ \begin{array}{cc|cc} 1 & 0 & 7 & -3 \\ 0 & 1 & -2 & 1 \end{array} \right].$$

So  $A^{-1} = \begin{bmatrix} 7 & -3 \\ -2 & 1 \end{bmatrix}$ , and multiplying out confirms  $AA^{-1} = I$ .

Two facts about inverses are used constantly. First, the inverse of a product reverses the order,

$$(AB)^{-1} = B^{-1}A^{-1},$$

which is checked by multiplying:  $ABB^{-1}A^{-1} = AIA^{-1} = I$ . Second, inversion commutes with transposition,  $(A^T)^{-1} = (A^{-1})^T$ .

**Remark 2.7.1** In numerical work one almost never forms  $A^{-1}$  explicitly. Solving  $Ax = b$  by elimination costs about a third as much as inverting  $A$  and is better behaved in floating point. The inverse is a tool for reasoning, not usually for computing.

Why the method works: the row operations amount to left multiplication by some matrix  $E$  with  $EA = I$ , and that identity says  $E = A^{-1}$ . The right-hand block simply records  $E$  by tracking what the same operations do to  $I$ .

## 2.8 The factorisation $A = LU$

Return to  $(E_{32}E_{21})A = U$  and move the elimination matrices to the other side:

$$A = E_{21}^{-1}E_{32}^{-1}U = LU.$$

The remarkable part is what  $L$  contains. Multiplying the two inverses,

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix},$$

whose subdiagonal entries are exactly the multipliers 3 and 2 used during elimination — in their original positions, with no interference between them. Verifying,

$$LU = \begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & -2 \\ 0 & 0 & 5 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 8 & 1 \\ 0 & 4 & 1 \end{bmatrix} = A.$$

**Theorem 2.8.1** *If elimination on  $A$  needs no row exchanges, then  $A = LU$  where  $U$  is the upper triangular matrix produced by elimination and  $L$  is lower triangular with ones on the diagonal and the multiplier  $\ell_{ij}$  in position  $(i, j)$ . With row exchanges, the same statement holds for a permuted matrix:  $PA = LU$ .*

Once  $A = LU$  is available, every subsequent right-hand side is cheap: solve  $Lc = b$  downwards, then  $Ux = c$  upwards. Both are triangular, so the expensive part of the work is done once and reused. A symmetric variant splits the pivots out as  $A = LDU$  with  $D$  diagonal, and for

The multipliers land in  $L$  unchanged only because the inverses are multiplied in the reverse order of the eliminations. Multiplying them in the forward order produces cross terms and destroys the pattern.



symmetric positive definite matrices it specialises to  $A = LL^T$ ; positive definiteness is taken up in Chapter 6.

## 2.9 Transposes and symmetry

The transpose exchanges rows and columns,  $(A^T)_{ij} = a_{ji}$ . Its two rules are

$$(AB)^T = B^T A^T, \quad (A^T)^T = A.$$

A matrix is *symmetric* when  $A^T = A$ . Symmetric matrices are not a curiosity: for any  $A$  whatsoever, both  $A^T A$  and  $AA^T$  are symmetric, since

$$(A^T A)^T = A^T (A^T)^T = A^T A.$$

$A^T A$  is the matrix behind least squares in Chapter 4 and behind the singular value decomposition in Chapter 7. Almost every time a rectangular matrix has to be turned into a square one, this is how it is done.

## 2.10 The cost of elimination

Clearing the first column of an  $n \times n$  matrix touches roughly  $n^2$  entries, the second about  $(n-1)^2$ , and so on, giving

$$n^2 + (n-1)^2 + \cdots + 1 \approx \frac{1}{3}n^3$$

operations for the factorisation, while each back-substitution costs about  $n^2$ . The cubic term is why reusing  $A = LU$  across many right-hand sides matters, and why the size of a linear solve grows uncomfortably fast — the practical reason that iterative and gradient-based methods dominate once  $n$  reaches the scale of a machine learning problem.

# The Four Fundamental Subspaces

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], §3.1–3.5; MIT 18.06 [7], Lectures 6–11.

Elimination answers a question about one right-hand side. The subspaces in this chapter answer the question for all right-hand sides at once, and they do it with four numbers: two dimensions in the input space and two in the output space.

## 3.1 Vector spaces and subspaces

**Definition 3.1.1** A subspace of  $\mathbb{R}^n$  is a non-empty set  $V \subseteq \mathbb{R}^n$  closed under addition and scalar multiplication: if  $\mathbf{u}, \mathbf{v} \in V$  and  $c \in \mathbb{R}$  then  $\mathbf{u} + \mathbf{v} \in V$  and  $c\mathbf{v} \in V$ .

Taking  $c = 0$  shows that every subspace contains  $\mathbf{0}$ , which is the quickest test to apply first. A plane through the origin in  $\mathbb{R}^3$  is a subspace; the same plane shifted away from the origin is not, since it fails that test.

The subspaces that matter here are not chosen arbitrarily. Each of the four is attached to a matrix and describes one aspect of what that matrix does.

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 3.1 Vector spaces and subspaces                                   | 13 |
| 3.2 The column space . . . . .                                    | 13 |
| 3.3 The nullspace . . . . .                                       | 13 |
| 3.4 Rank, echelon form, and $A = CR$ . . . . .                    | 14 |
| 3.5 The complete solution of $A\mathbf{x} = \mathbf{b}$ . . . . . | 15 |
| 3.6 Independence, basis, and dimension . . . . .                  | 15 |
| 3.7 The four subspaces . . . . .                                  | 15 |

Closure is the whole content of the definition. It says that the two operations the subject is built from cannot carry you out of the set — which is exactly what makes linear questions answerable inside it.

## 3.2 The column space

The column space  $\text{Col}(A) \subseteq \mathbb{R}^m$  was defined in Chapter 1 as the set of all combinations of the columns. It is a subspace because a combination of combinations is again a combination, and it settles solvability:  $A\mathbf{x} = \mathbf{b}$  has a solution precisely when  $\mathbf{b} \in \text{Col}(A)$ .

Throughout this chapter I use

$$A = \begin{bmatrix} 1 & 2 & 2 & 2 \\ 2 & 4 & 6 & 8 \\ 3 & 6 & 8 & 10 \end{bmatrix} \in \mathbb{R}^{3 \times 4}.$$

Its second column is twice its first, so the columns are dependent from the start and the column space is smaller than  $\mathbb{R}^3$ .

## 3.3 The nullspace

**Definition 3.3.1** The nullspace of  $A \in \mathbb{R}^{m \times n}$  is

$$\text{Null}(A) = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{0}\} \subseteq \mathbb{R}^n.$$

It is a subspace: if  $A\mathbf{u} = \mathbf{0}$  and  $A\mathbf{v} = \mathbf{0}$  then  $A(\mathbf{u} + c\mathbf{v}) = \mathbf{0}$ . Unlike the column space it lives in the *input* space, and it is exactly the object that Section 1.5 showed controls how many solutions a system has.

To compute it, eliminate. Subtracting  $2 \times \text{row 1}$  from row 2 and  $3 \times \text{row 1}$  from row 3, then subtracting the new row 2 from the new row 3,

$$A \rightarrow \begin{bmatrix} 1 & 2 & 2 & 2 \\ 0 & 0 & 2 & 4 \\ 0 & 0 & 2 & 4 \end{bmatrix} \rightarrow U = \begin{bmatrix} 1 & 2 & 2 & 2 \\ 0 & 0 & 2 & 4 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Two pivots appear, in columns 1 and 3. The row of zeros is not an accident: it records that row 3 of  $A$  was the sum of rows 1 and 2 all along.

**Definition 3.3.2** *Columns containing a pivot are pivot columns; the rest are free columns. The unknowns attached to free columns are free variables, because they may be assigned any values whatever, after which the pivot variables are determined.*

Here  $x_2$  and  $x_4$  are free. Setting  $x_2 = 1, x_4 = 0$  and back-substituting gives  $x_3 = 0$  and  $x_1 = -2$ ; setting  $x_2 = 0, x_4 = 1$  gives  $x_3 = -2$  and  $x_1 = 2$ . So

$$\mathbf{s}_1 = \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{s}_2 = \begin{bmatrix} 2 \\ 0 \\ -2 \\ 1 \end{bmatrix}, \quad \text{Null}(A) = \text{span}\{\mathbf{s}_1, \mathbf{s}_2\}.$$

Check both by hand.  $A\mathbf{s}_1 = -2\mathbf{a}_1 + \mathbf{a}_2$  and the second column is twice the first, so the result is  $\mathbf{0}$ . Similarly  $A\mathbf{s}_2 = 2\mathbf{a}_1 - 2\mathbf{a}_3 + \mathbf{a}_4 = \mathbf{0}$ .

### 3.4 Rank, echelon form, and $A = CR$

**Definition 3.4.1** *The rank  $r$  of  $A$  is the number of pivots produced by elimination.*

For the running example  $r = 2$ . Continuing elimination until each pivot is 1 and the entries above it are cleared produces the *reduced row echelon form*

$$R_0 = \begin{bmatrix} 1 & 2 & 0 & -2 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Discarding the zero row leaves  $R \in \mathbb{R}^{2 \times 4}$ , and collecting the pivot columns of the original matrix gives

$$C = \begin{bmatrix} 1 & 2 \\ 2 & 6 \\ 3 & 8 \end{bmatrix}, \quad R = \begin{bmatrix} 1 & 2 & 0 & -2 \\ 0 & 0 & 1 & 2 \end{bmatrix}.$$

**Theorem 3.4.1** *Every matrix factors as  $A = CR$ , where the  $r$  columns of  $C$  are the pivot columns of  $A$  and the  $r$  rows of  $R$  are the non-zero rows of the reduced row echelon form.*

Multiplying out confirms it here: the first column of  $CR$  is  $1 \cdot \mathbf{c}_1 = [1, 2, 3]^T$ , the second is  $2\mathbf{c}_1 = [2, 4, 6]^T$ , the third is  $\mathbf{c}_2 = [2, 6, 8]^T$ , and the fourth is  $-2\mathbf{c}_1 + 2\mathbf{c}_2 = [2, 8, 10]^T$ .

**Remark 3.4.1**  $A = CR$  is the cleanest statement of what rank means. The columns of  $A$  are combinations of  $r$  independent columns, and the rows of  $A$  are combinations of  $r$  independent rows — the *same*  $r$  in both cases. Row rank and column rank being equal is often presented as a surprise; here it is a consequence of a single factorisation.

### 3.5 The complete solution of $A\mathbf{x} = \mathbf{b}$

Take  $\mathbf{b} = [1, 6, 7]^T$ . Solving with the free variables set to zero gives a *particular* solution: the pivot variables must satisfy  $x_1\mathbf{a}_1 + x_3\mathbf{a}_3 = \mathbf{b}$ , that is  $x_1 + 2x_3 = 1$  and  $2x_1 + 6x_3 = 6$ , whence  $x_3 = 2$  and  $x_1 = -3$ ; the third equation  $3x_1 + 8x_3 = -9 + 16 = 7$  is consistent. So

$$\mathbf{x}_p = \begin{bmatrix} -3 \\ 0 \\ 2 \\ 0 \end{bmatrix}, \quad \mathbf{x} = \mathbf{x}_p + c_1\mathbf{s}_1 + c_2\mathbf{s}_2 \text{ for all } c_1, c_2 \in \mathbb{R}.$$

**Proposition 3.5.1** *If  $A\mathbf{x} = \mathbf{b}$  is solvable, its solution set is  $\mathbf{x}_p + \text{Null}(A)$ : one particular solution shifted by the whole nullspace.*

*Proof.* Any solution  $\mathbf{x}$  satisfies  $A(\mathbf{x} - \mathbf{x}_p) = \mathbf{b} - \mathbf{b} = \mathbf{0}$ , so  $\mathbf{x} - \mathbf{x}_p \in \text{Null}(A)$ . Conversely adding any nullspace vector to  $\mathbf{x}_p$  leaves the product unchanged.  $\square$

This is the same statement as the trichotomy in Chapter 1, now with the geometry supplied: the solution set is a shifted copy of a subspace, so it is empty, a single point, or an unbounded flat piece.

### 3.6 Independence, basis, and dimension

**Definition 3.6.1** *A basis of a subspace  $V$  is a set of vectors that is linearly independent and spans  $V$ . The number of vectors in a basis is the dimension of  $V$ , written  $\dim V$ .*

That the number is the same for every basis is the fact that makes dimension well defined. For the running example,  $\{\mathbf{s}_1, \mathbf{s}_2\}$  is a basis of the nullspace and the two pivot columns form a basis of the column space, so

$$\dim \text{Col}(A) = 2, \quad \dim \text{Null}(A) = 2.$$

### 3.7 The four subspaces

Applying the two constructions to  $A$  and to  $A^T$  yields four subspaces in total.

**Column space**  $\text{Col}(A) \subseteq \mathbb{R}^m$  spanned by the columns; dimension  $r$ .

**Nullspace**  $\text{Null}(A) \subseteq \mathbb{R}^n$  vectors killed by  $A$ ; dimension  $n - r$ .

**Row space**  $\text{Col}(A^T) \subseteq \mathbb{R}^n$  spanned by the rows; dimension  $r$ .

**Left nullspace**  $\text{Null}(A^T) \subseteq \mathbb{R}^m$  the vectors  $\mathbf{y}$  satisfying  $A^T\mathbf{y} = \mathbf{0}$ ; dimension  $m - r$ .

For the running example  $m = 3$ ,  $n = 4$ ,  $r = 2$ , so the dimensions are 2, 2, 2, 1. The left nullspace is spanned by  $[1, 1, -1]^T$ , which encodes the dependency noticed during elimination: row 1 plus row 2 equals row 3.

**Theorem 3.7.1** (Rank–nullity) For  $A \in \mathbb{R}^{m \times n}$  of rank  $r$ ,

$$\dim \text{Col}(A) + \dim \text{Null}(A) = r + (n - r) = n.$$

*Proof.* Each of the  $n$  columns is either a pivot column or a free column. The pivot columns form a basis of  $\text{Col}(A)$ , giving  $r$ ; each free column contributes exactly one special solution, and those are independent because each carries a 1 in a position where the others carry 0, giving  $n - r$ .  $\square$

**Remark 3.7.1** The counting has a plain reading. The input space  $\mathbb{R}^n$  splits its  $n$  degrees of freedom into  $r$  that the matrix records and  $n - r$  that it discards. Nothing else can happen, and no amount of cleverness in choosing  $\mathbf{b}$  will change the split.

Chapter 4 sharpens this picture considerably: the two subspaces inside  $\mathbb{R}^n$  are not merely complementary in dimension, they are orthogonal complements, and likewise for the two inside  $\mathbb{R}^m$ .

# Orthogonality, Projection, and Least Squares

# 4

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], §4.1–4.4; MIT 18.06 [7], Lectures 14–17.

Most systems that arise from measurements have no solution: there are more equations than unknowns and the data are noisy. Rather than give up, one asks for the vector that comes closest. Orthogonality is what makes “closest” computable.

|                                                  |    |
|--------------------------------------------------|----|
| 4.1 Inner products, length, and angle . . . . .  | 17 |
| 4.2 Orthogonal subspaces . . .                   | 17 |
| 4.3 Projection onto a line . . .                 | 18 |
| 4.4 Projection onto a subspace                   | 18 |
| 4.5 Least squares . . . . .                      | 18 |
| 4.6 Orthonormal bases and Gram–Schmidt . . . . . | 19 |
| 4.7 The factorisation $A = QR$ .                 | 19 |

## 4.1 Inner products, length, and angle

For  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$  the inner product is  $\mathbf{u}^T \mathbf{v} = \sum_i u_i v_i$ , the length is  $\|\mathbf{v}\| = \sqrt{\mathbf{v}^T \mathbf{v}}$ , and the two vectors are *orthogonal* when  $\mathbf{u}^T \mathbf{v} = 0$ .

**Proposition 4.1.1**  $\mathbf{u}^T \mathbf{v} = 0$  if and only if  $\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$ .

*Proof.* Expanding,  $\|\mathbf{u} + \mathbf{v}\|^2 = (\mathbf{u} + \mathbf{v})^T (\mathbf{u} + \mathbf{v}) = \|\mathbf{u}\|^2 + 2\mathbf{u}^T \mathbf{v} + \|\mathbf{v}\|^2$ . The two sides agree exactly when the cross term vanishes.  $\square$

So Pythagoras is not an analogy imported from geometry; it is the algebraic identity above, and it holds in  $\mathbb{R}^n$  for every  $n$ .

## 4.2 Orthogonal subspaces

Two subspaces  $V, W$  are orthogonal when  $\mathbf{v}^T \mathbf{w} = 0$  for every  $\mathbf{v} \in V$  and  $\mathbf{w} \in W$ . The four subspaces of Chapter 3 pair up in exactly this way.

**Theorem 4.2.1** For any  $A \in \mathbb{R}^{m \times n}$ , the nullspace and the row space are orthogonal complements in  $\mathbb{R}^n$ , and the left nullspace and the column space are orthogonal complements in  $\mathbb{R}^m$ .

*Proof.* If  $A\mathbf{x} = \mathbf{0}$  then every row of  $A$  has zero inner product with  $\mathbf{x}$ , hence so does every combination of rows:  $\text{Null}(A)$  is orthogonal to  $\text{Col}(A^T)$ . Their dimensions are  $n - r$  and  $r$ , which sum to  $n$ , so neither can be enlarged inside the other’s complement. Applying the same argument to  $A^T$  gives the second pair.  $\square$

**Remark 4.2.1** This upgrades rank–nullity from an accounting identity to a geometric statement. The input space splits into a part the matrix reproduces faithfully and a part it annihilates, and the two parts meet at right angles.

### 4.3 Projection onto a line

Given  $\mathbf{a} \neq \mathbf{0}$ , the point of the line  $\{c\mathbf{a}\}$  closest to  $\mathbf{b}$  is the one for which the error  $\mathbf{b} - c\mathbf{a}$  is perpendicular to  $\mathbf{a}$ :

$$\mathbf{a}^\top(\mathbf{b} - c\mathbf{a}) = 0 \implies c = \frac{\mathbf{a}^\top \mathbf{b}}{\mathbf{a}^\top \mathbf{a}}, \quad \mathbf{p} = \frac{\mathbf{a}^\top \mathbf{b}}{\mathbf{a}^\top \mathbf{a}} \mathbf{a}.$$

Writing this as a matrix acting on  $\mathbf{b}$  gives the projection matrix

$$P = \frac{\mathbf{a}\mathbf{a}^\top}{\mathbf{a}^\top \mathbf{a}}, \quad P^2 = P, \quad P^\top = P.$$

$P^2 = P$  says projecting twice is projecting once, which any sensible notion of “closest point” must satisfy. Symmetry is the algebraic trace of the right angle.

**Example 4.3.1** With  $\mathbf{a} = [1, 1, 1]^\top$  and  $\mathbf{b} = [1, 2, 2]^\top$ ,  $\mathbf{a}^\top \mathbf{b} = 5$  and  $\mathbf{a}^\top \mathbf{a} = 3$ , so  $\mathbf{p} = \frac{5}{3}[1, 1, 1]^\top$  and the error  $\mathbf{b} - \mathbf{p} = \frac{1}{3}[-2, 1, 1]^\top$  indeed satisfies  $\mathbf{a}^\top(\mathbf{b} - \mathbf{p}) = 0$ .

### 4.4 Projection onto a subspace

Replace the single vector by the columns of  $A \in \mathbb{R}^{m \times n}$  and look for  $\hat{\mathbf{x}}$  making  $\mathbf{b} - A\hat{\mathbf{x}}$  orthogonal to every column. That is  $A^\top(\mathbf{b} - A\hat{\mathbf{x}}) = \mathbf{0}$ , or

$$A^\top A \hat{\mathbf{x}} = A^\top \mathbf{b}, \tag{4.1}$$

the *normal equations*. When the columns of  $A$  are independent,  $A^\top A$  is invertible and

$$\hat{\mathbf{x}} = (A^\top A)^{-1} A^\top \mathbf{b}, \quad \mathbf{p} = A\hat{\mathbf{x}} = A(A^\top A)^{-1} A^\top \mathbf{b}.$$

**Proposition 4.4.1** *If the columns of  $A$  are linearly independent then  $A^\top A$  is invertible.*

*Proof.* Suppose  $A^\top A \mathbf{x} = \mathbf{0}$ . Then  $\mathbf{x}^\top A^\top A \mathbf{x} = \|A\mathbf{x}\|^2 = 0$ , so  $A\mathbf{x} = \mathbf{0}$ , and independence of the columns forces  $\mathbf{x} = \mathbf{0}$ . A square matrix with trivial nullspace is invertible.  $\square$

The proof is the reason  $A^\top A$  appears everywhere. It converts a rectangular problem into a square one without losing information, provided the columns were independent to begin with.

### 4.5 Least squares

Solving Equation 4.1 minimises  $\|A\mathbf{x} - \mathbf{b}\|^2$  over all  $\mathbf{x}$ . This is the standard way to fit a model with more observations than parameters.

**Example 4.5.1** Fit a straight line  $b = C + Dt$  through the points  $(1, 1)$ ,  $(2, 2)$ ,  $(3, 2)$ . Then

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \quad A^\top A = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}, \quad A^\top \mathbf{b} = \begin{bmatrix} 5 \\ 11 \end{bmatrix}.$$

The normal equations  $3C + 6D = 5$  and  $6C + 14D = 11$  give  $D = \frac{1}{2}$  and  $C = \frac{2}{3}$ , so the best line is  $b = \frac{2}{3} + \frac{1}{2}t$ . The residuals are  $-\frac{1}{6}, \frac{1}{3}, -\frac{1}{6}$ ; they sum to zero and satisfy  $A^T \mathbf{e} = \mathbf{0}$ , confirming that the error is orthogonal to the column space.

**Remark 4.5.1** Least squares is the first place where a linear algebra identity and a minimisation problem coincide. The same coincidence returns in Chapter 9 for quadratic objectives and again in Chapter 19, where minimising squared error turns out to be maximum likelihood under Gaussian noise.

## 4.6 Orthonormal bases and Gram–Schmidt

A set  $\{\mathbf{q}_1, \dots, \mathbf{q}_n\}$  is *orthonormal* when  $\mathbf{q}_i^T \mathbf{q}_j$  equals 1 for  $i = j$  and 0 otherwise. Collecting them as columns of  $Q$  gives  $Q^T Q = I$ , and when  $Q$  is square this is  $Q^{-1} = Q^T$ .

Orthonormal columns simplify everything above: the normal equations collapse to  $\hat{\mathbf{x}} = Q^T \mathbf{b}$ , and the projection matrix becomes  $P = QQ^T$ , with no inverse to compute.

Gram–Schmidt manufactures an orthonormal basis from any independent set by subtracting, from each vector in turn, its projection onto everything already accepted.

Numerically this is the difference between a stable computation and an unstable one. Forming  $A^T A$  squares the condition number of  $A$ ; orthogonalising first avoids that.

**Example 4.6.1** For  $A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$ , start with  $\mathbf{q}_1 = \frac{1}{\sqrt{2}}[1, 1, 0]^T$ . Subtracting the projection of the second column,

$$\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ 1 \end{bmatrix}, \quad \mathbf{q}_2 = \frac{1}{\sqrt{6}} \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix}.$$

## 4.7 The factorisation $A = QR$

Recording the coefficients used during Gram–Schmidt gives a second factorisation alongside  $A = LU$ .

**Theorem 4.7.1** If  $A \in \mathbb{R}^{m \times n}$  has independent columns then  $A = QR$ , where the columns of  $Q \in \mathbb{R}^{m \times n}$  are orthonormal and  $R \in \mathbb{R}^{n \times n}$  is upper triangular with positive diagonal entries.

For the example above,

$$R = \begin{bmatrix} \sqrt{2} & \frac{1}{\sqrt{2}} \\ 0 & \sqrt{\frac{3}{2}} \end{bmatrix},$$

and multiplying  $QR$  returns the two original columns. Substituting  $A = QR$  into the normal equations turns them into the triangular system  $R\hat{\mathbf{x}} = Q^T \mathbf{b}$ , which is how least squares is solved in practice.



**Remark 4.7.1** Two factorisations, two purposes.  $A = LU$  records elimination and answers questions about solving;  $A = QR$  records orthogonalisation and answers questions about approximating. Chapter 7 produces a third that does both and survives rank deficiency as well.

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], §5.1–5.3; MIT 18.06 [7], Lectures 18–20.

The determinant compresses a square matrix into one number that answers a yes-or-no question: is the matrix invertible? Its main use in this book is as the machinery behind eigenvalues in Chapter 6.

|                                               |    |
|-----------------------------------------------|----|
| 5.1 Three properties that define it . . . . . | 21 |
| 5.2 What follows immediately . . . . .        | 21 |
| 5.3 Determinant from the pivots . . . . .     | 22 |
| 5.4 Cofactors and the big formula . . . . .   | 22 |
| 5.5 Three uses . . . . .                      | 23 |

## 5.1 Three properties that define it

Rather than start from a formula, take three requirements and derive everything from them. For  $A \in \mathbb{R}^{n \times n}$ , the determinant  $\det A$  is the unique function satisfying:

1.  $\det I = 1$ ;
2. exchanging two rows reverses the sign;
3. it is linear in each row separately, the other rows being held fixed.

Property 3 is linearity *one row at a time*, not linearity in the matrix. Multiplying the whole of an  $n \times n$  matrix by  $c$  multiplies the determinant by  $c^n$ , once per row.

## 5.2 What follows immediately

**Proposition 5.2.1** *If two rows of  $A$  are equal then  $\det A = 0$ .*

*Proof.* Exchanging the two equal rows leaves  $A$  unchanged but reverses the sign by property 2, so  $\det A = -\det A$ .  $\square$

**Proposition 5.2.2** *Subtracting a multiple of one row from another leaves the determinant unchanged.*

*Proof.* By linearity in the affected row, the determinant of the result is  $\det A$  minus  $c$  times the determinant of a matrix with two equal rows, and that second term is zero.  $\square$

These two facts make elimination a legitimate way to compute a determinant: the subtraction steps cost nothing and only the row exchanges have to be counted.

**Proposition 5.2.3** *A triangular matrix has determinant equal to the product of its diagonal entries; a matrix with a zero row has determinant 0.*

### 5.3 Determinant from the pivots

Combining the last two sections: eliminate  $A$  into  $U$ , keeping track of how many row exchanges were used. Then

$$\det A = (-1)^{\text{number of exchanges}} \prod_{i=1}^n u_{ii},$$

the product of the pivots up to sign.

**Example 5.3.1** The elimination carried out in Section 2.1 on

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 8 & 1 \\ 0 & 4 & 1 \end{bmatrix}$$

needed no row exchanges and produced the pivots 1, 2, 5, so  $\det A = 10$ .

**Theorem 5.3.1**  $A$  is invertible if and only if  $\det A \neq 0$ .

*Proof.* Elimination produces  $n$  non-zero pivots exactly when  $A$  is non-singular, and the displayed formula makes  $\det A$  non-zero precisely in that case.  $\square$

The multiplicative rule  $\det(AB) = (\det A)(\det B)$  follows from the same circle of ideas, and gives  $\det(A^{-1}) = 1/\det A$  as a special case. Transposition also leaves it alone,  $\det(A^T) = \det A$ , so each statement about rows has a counterpart about columns.

### 5.4 Cofactors and the big formula

Expanding property 3 across all rows at once gives a sum over the  $n!$  permutations, one term for each way of picking a single entry from every row and column. For  $n = 3$  this is the familiar six-term expression; for  $n = 10$  it has over three million terms and is useless for computation.

The practical repackaging is cofactor expansion along a row,

$$\det A = \sum_{j=1}^n a_{ij} C_{ij}, \quad C_{ij} = (-1)^{i+j} \det M_{ij},$$

where  $M_{ij}$  deletes row  $i$  and column  $j$ .

Two routes, one answer. Cofactors are convenient for small or sparse matrices and for symbolic work; pivots are what any implementation actually uses.

**Example 5.4.1** Expanding the same matrix along its first row,

$$\det A = 1(8 - 4) - 2(3 - 0) + 1(12 - 0) = 4 - 6 + 12 = 10,$$

agreeing with the pivot calculation.

## 5.5 Three uses

**The inverse formula.**  $A^{-1} = \frac{1}{\det A} C^T$ , where  $C$  is the matrix of cofactors. It makes the dependence of  $A^{-1}$  on the entries of  $A$  explicit, and shows why a determinant near zero makes the inverse enormous.

**Cramer's rule.** The  $j$ th component of the solution of  $A\mathbf{x} = \mathbf{b}$  is  $x_j = \det B_j / \det A$ , where  $B_j$  is  $A$  with column  $j$  replaced by  $\mathbf{b}$ . Elegant, and far too slow to use.

**Volume.**  $|\det A|$  is the volume of the parallelepiped spanned by the rows of  $A$ . A zero determinant means the solid is flat — the rows lie in a lower dimensional subspace — which is the geometric form of singularity.

**Remark 5.5.1** The volume reading is the one that carries into calculus. In Chapter 13 the determinant of the Jacobian is the local volume-scaling factor of a nonlinear map, and it is what appears when a change of variables is made inside an integral.

# 6

## Eigenvalues and Eigenvectors

|                                  |    |
|----------------------------------|----|
| 6.1 The defining equation . . .  | 24 |
| 6.2 Trace and determinant . . .  | 24 |
| 6.3 Diagonalisation and powers   | 25 |
| 6.4 Symmetric matrices . . . .   | 25 |
| 6.5 Positive definite matrices . | 26 |
| 6.6 Similar matrices . . . . .   | 26 |

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], §6.1–6.5; MIT 18.06 [7], Lectures 21–22 and 26–28.

Everything so far treated  $A$  as a machine for solving. This chapter asks a different question: are there directions in which  $A$  does nothing more interesting than stretching? The answer reorganises the matrix around those directions and makes repeated application easy to describe.

### 6.1 The defining equation

**Definition 6.1.1** A non-zero vector  $\mathbf{x}$  is an eigenvector of  $A \in \mathbb{R}^{n \times n}$  with eigenvalue  $\lambda$  when

$$A\mathbf{x} = \lambda\mathbf{x}.$$

Rewriting as  $(A - \lambda I)\mathbf{x} = \mathbf{0}$  shows that  $\lambda$  is an eigenvalue exactly when  $A - \lambda I$  is singular, since a non-zero vector must sit in its nullspace. By Chapter 5 that is the condition

$$\det(A - \lambda I) = 0,$$

the *characteristic equation*. Its left-hand side is a polynomial of degree  $n$  in  $\lambda$ .

Note the order of work: first find the  $\lambda$  that make the matrix singular, then find the nullspace for each. Eigenvectors are never found first.

**Example 6.1.1** For  $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ ,

$$\det \begin{bmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{bmatrix} = (2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3,$$

with roots  $\lambda_1 = 3$  and  $\lambda_2 = 1$ . Solving  $(A - 3I)\mathbf{x} = \mathbf{0}$  gives  $\mathbf{x}_1 = [1, 1]^T$ , and  $(A - I)\mathbf{x} = \mathbf{0}$  gives  $\mathbf{x}_2 = [1, -1]^T$ . The two eigenvectors are orthogonal, which Section 6.4 explains.

### 6.2 Trace and determinant

Two shortcuts follow from comparing coefficients in the characteristic polynomial.

**Proposition 6.2.1** The eigenvalues of  $A$ , counted with multiplicity, satisfy

$$\sum_i \lambda_i = \operatorname{tr} A = \sum_i a_{ii}, \quad \prod_i \lambda_i = \det A.$$

For the example,  $3 + 1 = 4 = 2 + 2$  and  $3 \cdot 1 = 3 = 4 - 1$ . In two dimensions these two equations determine the eigenvalues outright, which is often faster than forming the polynomial.

**Remark 6.2.1** Singularity and a zero eigenvalue are the same statement, since the product of the eigenvalues is the determinant. The nullspace of  $A$  is precisely the eigenspace for  $\lambda = 0$ .

## 6.3 Diagonalisation and powers

Suppose  $A$  has  $n$  independent eigenvectors. Placing them in the columns of  $X$  and the eigenvalues on the diagonal of  $\Lambda$ ,

$$AX = X\Lambda \implies A = X\Lambda X^{-1}.$$

The payoff is that powers telescope:

$$A^k = X\Lambda^k X^{-1},$$

because the inner  $X^{-1}X$  factors cancel. Raising a diagonal matrix to a power is done entry by entry, so the long-run behaviour of  $A^k$  is decided entirely by the sizes  $|\lambda_i|$ .

**Example 6.3.1** Let  $A = \begin{bmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{bmatrix}$ , whose columns sum to 1. Its trace is 1.5 and its determinant is 0.5, so the eigenvalues solve  $\lambda^2 - 1.5\lambda + 0.5 = 0$ , giving  $\lambda_1 = 1$  and  $\lambda_2 = 0.5$ . The eigenvector for  $\lambda_1 = 1$  is  $[3, 2]^T$ ; normalising it to sum to 1 gives  $[0.6, 0.4]^T$ . Since  $0.5^k \rightarrow 0$ , every starting distribution converges to that vector.

This is the entire theory of Markov chains in miniature: an eigenvalue of exactly 1 supplies the steady state, and the second largest modulus sets the rate of approach.

Diagonalisation can fail. The matrix  $\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$  has  $\lambda = 0$  twice but only a one-dimensional eigenspace, so no basis of eigenvectors exists. Distinct eigenvalues guarantee independence and therefore diagonalisability; repeated eigenvalues may or may not.

## 6.4 Symmetric matrices

Symmetric matrices are the well-behaved case, and they are the case that occurs in practice — covariance matrices, Gram matrices  $A^T A$ , and Hessians are all symmetric.

**Theorem 6.4.1** (Spectral theorem) *If  $A^T = A$  then all eigenvalues of  $A$  are real, eigenvectors for distinct eigenvalues are orthogonal, and  $A$  can be written*

$$A = Q\Lambda Q^T$$

*with  $Q$  orthogonal and  $\Lambda$  real diagonal.*

*Orthogonality.* Let  $Ax = \lambda x$  and  $Ay = \mu y$  with  $\lambda \neq \mu$ . Then

$$\lambda y^T x = y^T Ax = (Ay)^T x = \mu y^T x,$$

using symmetry in the middle step. Hence  $(\lambda - \mu)y^T x = 0$ , and since  $\lambda \neq \mu$  the inner product vanishes.  $\square$

Because  $Q^{-1} = Q^T$ , the factorisation writes  $A$  as a sum of orthogonal rank-one pieces,

$$A = \sum_{i=1}^n \lambda_i \mathbf{q}_i \mathbf{q}_i^T,$$

which is the column-times-row reading of Section 2.4, applied here to the basis of eigenvectors.

## 6.5 Positive definite matrices

**Definition 6.5.1** A symmetric  $A$  is positive definite when  $\mathbf{x}^T A \mathbf{x} > 0$  for every  $\mathbf{x} \neq \mathbf{0}$ , and positive semidefinite when the inequality is weak.

**Proposition 6.5.1** For a symmetric matrix  $A$  the following are equivalent: every eigenvalue is positive; every pivot is positive;  $\mathbf{x}^T A \mathbf{x} > 0$  for every  $\mathbf{x} \neq \mathbf{0}$ ; and  $A = B^T B$  for some  $B$  with independent columns.

**Example 6.5.1**  $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$  has eigenvalues 3 and 1, pivots 2 and  $\frac{3}{2}$ , and  $\mathbf{x}^T A \mathbf{x} = 2x_1^2 + 2x_1x_2 + 2x_2^2 = (x_1 + x_2)^2 + x_1^2 + x_2^2$ , positive unless both components vanish. All three tests agree.

Positive definiteness is the matrix version of “strictly convex”. Chapter 14 uses exactly this: a quadratic with a positive definite Hessian has a single minimum, and the ratio of largest to smallest eigenvalue predicts how slowly gradient descent will find it.

## 6.6 Similar matrices

Matrices  $A$  and  $M^{-1}AM$  are called *similar*. They have the same characteristic polynomial and hence the same eigenvalues, because

$$\det(M^{-1}AM - \lambda I) = \det(M^{-1}(A - \lambda I)M) = \det(A - \lambda I).$$

Similarity is a change of basis, so the message is that eigenvalues belong to the underlying transformation rather than to the particular array of numbers used to record it — a point taken up properly in Chapter 8.

# The Singular Value Decomposition

# 7

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], §7.1–7.4; MIT 18.06 [7], Lecture 31.

Eigenvalues require a square matrix and may fail to supply a full basis. The singular value decomposition has neither restriction: every matrix has one. It is the factorisation that ties the four subspaces together and the one that underlies most of Chapter 10.

|                                                      |    |
|------------------------------------------------------|----|
| 7.1 The statement . . . . .                          | 27 |
| 7.2 Where it comes from . . . .                      | 27 |
| 7.3 The four subspaces, one picture . . . . .        | 28 |
| 7.4 Rank-one pieces and best approximation . . . . . | 28 |
| 7.5 The pseudoinverse . . . . .                      | 28 |

## 7.1 The statement

**Theorem 7.1.1** Every  $A \in \mathbb{R}^{m \times n}$  can be written

$$A = U\Sigma V^T,$$

where  $U \in \mathbb{R}^{m \times m}$  and  $V \in \mathbb{R}^{n \times n}$  are orthogonal and  $\Sigma \in \mathbb{R}^{m \times n}$  is diagonal with entries  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$  and zeros elsewhere,  $r$  being the rank of  $A$ .

The columns  $\mathbf{v}_j$  are the *right singular vectors*, the columns  $\mathbf{u}_i$  the *left singular vectors*, and the  $\sigma_i$  the *singular values*. The content of the factorisation is the pairing

$$A\mathbf{v}_i = \sigma_i \mathbf{u}_i, \quad i = 1, \dots, r,$$

which says that  $A$  maps one orthonormal set to another, up to individual stretching factors.

Compare with eigenvectors, where the same basis appears on both sides. Allowing two different bases is exactly what removes the need for  $A$  to be square or diagonalisable.

## 7.2 Where it comes from

Suppose  $A = U\Sigma V^T$ . Then

$$A^T A = V\Sigma^T U^T U \Sigma V^T = V(\Sigma^T \Sigma) V^T, \quad AA^T = U(\Sigma \Sigma^T) U^T.$$

Both are symmetric and positive semidefinite, so Section 6.4 applies. The  $\mathbf{v}_i$  are eigenvectors of  $A^T A$ , the  $\mathbf{u}_i$  are eigenvectors of  $AA^T$ , and the singular values are the square roots of the shared non-zero eigenvalues.

**Example 7.2.1** Let  $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$ . Then  $A^T A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$ , with eigenvalues 45 and 5 and eigenvectors  $[1, 1]^T$  and  $[1, -1]^T$ . Hence  $\sigma_1 = 3\sqrt{5}$ ,  $\sigma_2 = \sqrt{5}$ , and

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{u}_1 = \frac{A\mathbf{v}_1}{\sigma_1} = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad \mathbf{u}_2 = \frac{A\mathbf{v}_2}{\sigma_2} = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ -1 \end{bmatrix}.$$

As a check,  $\mathbf{u}_1^T \mathbf{u}_2 = 0$  and  $\sigma_1 \sigma_2 = 3\sqrt{5} \cdot \sqrt{5} = 15 = \det A$ .

**Remark 7.2.1** In floating point one does not form  $A^T A$ ; squaring the



matrix squares its condition number. The derivation above explains what the singular vectors are; production algorithms compute them by orthogonal transformations applied to  $A$  directly.

### 7.3 The four subspaces, one picture

Split the singular vectors at index  $r$ :

$$\underbrace{\mathbf{v}_1, \dots, \mathbf{v}_r}_{\text{row space}} \quad \underbrace{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n}_{\text{Null}(A)} \quad \underbrace{\mathbf{u}_1, \dots, \mathbf{u}_r}_{\text{Col}(A)} \quad \underbrace{\mathbf{u}_{r+1}, \dots, \mathbf{u}_m}_{\text{Null}(A^\top)}$$

Each group is an orthonormal basis of the subspace beneath it. The orthogonality statements of Chapter 4 are then read off directly, and the action of  $A$  becomes: discard the nullspace component, and stretch the surviving  $r$  directions by  $\sigma_1, \dots, \sigma_r$ .

### 7.4 Rank-one pieces and best approximation

The column-times-row reading of Section 2.4 turns the factorisation into a sum ordered by importance:

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top. \quad (7.1)$$

Truncating after  $k$  terms gives  $A_k$ , and no other matrix of rank  $k$  comes closer.

**Theorem 7.4.1** (Eckart–Young) *Among all matrices  $B$  of rank at most  $k$ , the truncation  $A_k$  minimises  $\|A - B\|$  in both the spectral and the Frobenius norm, with  $\|A - A_k\|_2 = \sigma_{k+1}$ .*

This single theorem is the justification for principal component analysis, low-rank compression of weight matrices, and latent-factor models alike. They differ in what the matrix contains, not in the mathematics applied to it.

### 7.5 The pseudoinverse

When  $A$  is not invertible, the closest available substitute is

$$A^+ = V \Sigma^+ U^\top,$$

where  $\Sigma^+$  transposes  $\Sigma$  and replaces each  $\sigma_i > 0$  by  $1/\sigma_i$ , leaving the zeros alone. Then  $\mathbf{x} = A^+ \mathbf{b}$  is the least-squares solution of smallest length: among all vectors minimising  $\|A\mathbf{x} - \mathbf{b}\|$ , it is the one lying entirely in the row space.

**Remark 7.5.1** This closes a loop opened in Chapter 4. There the normal equations needed independent columns so that  $A^\top A$  could be inverted. The pseudoinverse removes that condition and picks a canonical answer from the infinitely many that rank deficiency allows.

# Linear Transformations and Change of Basis

# 8

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], §8.1–8.3; MIT 18.06 [7], Lectures 33–34.

A matrix is a table of numbers; a linear transformation is a rule. This chapter separates the two, which explains why the same rule can be recorded by many different matrices and why choosing the basis well is worth the trouble.

|                                              |    |
|----------------------------------------------|----|
| 8.1 What makes a map linear .                | 29 |
| 8.2 The matrix of a transformation . . . . . | 29 |
| 8.3 Change of basis . . . . .                | 30 |
| 8.4 Choosing the basis well . .              | 30 |

## 8.1 What makes a map linear

**Definition 8.1.1** A map  $T : V \rightarrow W$  between vector spaces is linear when

$$T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v}), \quad T(c\mathbf{v}) = cT(\mathbf{v})$$

for all  $\mathbf{u}, \mathbf{v} \in V$  and scalars  $c$ .

Setting  $c = 0$  gives  $T(\mathbf{0}) = \mathbf{0}$ , so any map that shifts the origin is disqualified. Rotation about the origin, projection onto a subspace, and differentiation of polynomials are linear; translation and the map  $\mathbf{v} \mapsto \|\mathbf{v}\|$  are not.

The two conditions combine into one:  $T$  is linear exactly when it commutes with taking linear combinations. Everything in this book is a consequence of that single permission.

## 8.2 The matrix of a transformation

Fix a basis  $\mathbf{v}_1, \dots, \mathbf{v}_n$  of  $V$  and a basis  $\mathbf{w}_1, \dots, \mathbf{w}_m$  of  $W$ . Because  $T$  respects combinations, it is determined by what it does to the basis vectors. Writing each image in the output basis,

$$T(\mathbf{v}_j) = \sum_{i=1}^m a_{ij} \mathbf{w}_i,$$

collects the coefficients into a matrix  $A \in \mathbb{R}^{m \times n}$ , and the rule becomes matrix multiplication in coordinates.

**Remark 8.2.1** Column  $j$  of  $A$  is nothing other than the coordinate vector of  $T(\mathbf{v}_j)$ . Building the matrix of a transformation is always the same mechanical act: feed in each basis vector, write the answer in the output basis, and stack the results as columns.

**Example 8.2.1** Let  $V$  be the polynomials of degree at most 3, with basis  $1, t, t^2, t^3$ , and let  $W$  be those of degree at most 2, with basis  $1, t, t^2$ . Differentiation sends  $1 \mapsto 0, t \mapsto 1, t^2 \mapsto 2t, t^3 \mapsto 3t^2$ , so

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix}.$$

Its nullspace is spanned by  $[1, 0, 0, 0]^T$ , the constants — exactly the poly-

nomials that differentiate to zero — and its rank is 3, matching  $3 + 1 = 4$  in Chapter 3.

**Example 8.2.2** Rotation of  $\mathbb{R}^2$  by an angle  $\theta$  sends  $\mathbf{e}_1$  to  $(\cos \theta, \sin \theta)$  and  $\mathbf{e}_2$  to  $(-\sin \theta, \cos \theta)$ , so

$$A = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix},$$

which satisfies  $A^\top A = I$ : rotations preserve length, as an orthogonal matrix must.

## 8.3 Change of basis

Keeping the transformation and changing the basis changes the matrix. Let  $M$  have as its columns the new basis vectors expressed in the old basis. A vector with new coordinates  $\mathbf{c}$  has old coordinates  $M\mathbf{c}$ , and the two matrices of the same transformation are related by

$$A_{\text{new}} = M^{-1}A_{\text{old}}M.$$

This is the similarity relation of Section 6.6, now with its meaning supplied: similar matrices are the same transformation described by two observers using different rulers. Eigenvalues, trace, determinant and rank are properties of the transformation and are therefore identical for all of them; the individual entries are not.

A useful sanity check when a formula looks wrong: ask whether the quantity you computed could possibly depend on the choice of basis. If it should not, and your expression does, the error is in the bookkeeping.

## 8.4 Choosing the basis well

Diagonalisation is the statement that, for a diagonalisable  $A$ , the eigenvector basis makes the matrix diagonal:  $\Lambda = X^{-1}AX$ . In that basis the transformation is  $n$  independent scalings, and questions about powers, exponentials, and long-run behaviour become questions about numbers.

When  $A$  is not square, or not diagonalisable, the singular value decomposition supplies the substitute. Taking  $\mathbf{v}_1, \dots, \mathbf{v}_n$  as the input basis and  $\mathbf{u}_1, \dots, \mathbf{u}_m$  as the output basis, the matrix of the transformation is  $\Sigma$  — diagonal, non-negative, and ordered. No two-basis choice does better.

**Remark 8.4.1** Read this way, the factorisations of Part I are all answers to one question: *which coordinates make this transformation simple?* Elimination answers it for solving, Gram–Schmidt for projecting, diagonalisation for iterating, and the singular value decomposition for approximating.

# Linear Algebra in Optimisation

# 9

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], Chapter 9.

Optimisation problems that can be solved exactly are, with few exceptions, quadratic ones. This chapter collects the linear algebra that governs them: the norms used to measure error, the condition number that predicts difficulty, and the quadratic form whose minimiser is a linear solve in disguise. The calculus and the algorithms come in Chapter 14.

|                               |    |
|-------------------------------|----|
| 9.1 Norms . . . . .           | 31 |
| 9.2 Conditioning . . . . .    | 31 |
| 9.3 Quadratic forms . . . . . | 31 |
| 9.4 Regularisation . . . . .  | 32 |

## 9.1 Norms

For vectors the three standard choices are

$$\|\mathbf{x}\|_1 = \sum_i |x_i|, \quad \|\mathbf{x}\|_2 = \left( \sum_i x_i^2 \right)^{1/2}, \quad \|\mathbf{x}\|_\infty = \max_i |x_i|.$$

All three satisfy the triangle inequality; only  $\|\cdot\|_2$  comes from an inner product, which is why projection and least squares are stated in it.

For matrices, two norms are used constantly:

$$\|A\|_2 = \sigma_1, \quad \|A\|_F = \left( \sum_{i,j} a_{ij}^2 \right)^{1/2} = \left( \sum_i \sigma_i^2 \right)^{1/2},$$

the largest singular value and the Frobenius norm. Both are supplied by the decomposition of Chapter 7; the second identity says the Frobenius norm does not care which orthonormal basis the matrix is written in.

The choice is not cosmetic. Penalising  $\|\cdot\|_1$  drives coefficients exactly to zero and produces sparse solutions; penalising  $\|\cdot\|_2$  shrinks them smoothly and keeps them non-zero.

## 9.2 Conditioning

**Definition 9.2.1** The condition number of an invertible  $A$  is  $\kappa(A) = \sigma_1/\sigma_n$ , the ratio of largest to smallest singular value.

It bounds how much a relative perturbation of  $\mathbf{b}$  can be amplified in the solution of  $A\mathbf{x} = \mathbf{b}$ : a relative change of size  $\varepsilon$  in the data produces a relative change of at most  $\kappa(A)\varepsilon$  in the answer. A matrix with  $\kappa = 10^8$  loses roughly eight digits, whatever algorithm is used.

**Remark 9.2.1** This is the precise reason Chapter 4 warned against forming  $A^T A$ . Its singular values are the squares of those of  $A$ , so  $\kappa(A^T A) = \kappa(A)^2$  and the difficulty is squared along with them.

## 9.3 Quadratic forms

A quadratic function on  $\mathbb{R}^n$  can always be written

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x} + c,$$

with  $A$  symmetric — any non-symmetric part cancels in the product and can be discarded without changing  $f$ .

**Theorem 9.3.1** *If  $A$  is symmetric positive definite then  $f$  has a unique minimiser, and it is the solution of  $A\mathbf{x} = \mathbf{b}$ .*

*Proof.* Let  $A\mathbf{x}^* = \mathbf{b}$  and write  $\mathbf{x} = \mathbf{x}^* + \mathbf{d}$ . Expanding and using symmetry,

$$f(\mathbf{x}) = f(\mathbf{x}^*) + \frac{1}{2}\mathbf{d}^T A \mathbf{d},$$

and positive definiteness makes the last term strictly positive unless  $\mathbf{d} = \mathbf{0}$ .  $\square$

Minimisation and solving are therefore the same problem wearing two hats. Least squares in Chapter 4 was the first instance:  $\|A\mathbf{x} - \mathbf{b}\|^2$  is a quadratic form whose matrix is  $A^T A$ .

The level sets of  $f$  are ellipsoids whose axes are the eigenvectors of  $A$ , with lengths proportional to  $1/\sqrt{\lambda_i}$ . A large ratio  $\lambda_{\max}/\lambda_{\min}$  makes them long and thin, which is exactly the geometry that slows gradient methods down in Chapter 14.

## 9.4 Regularisation

When  $A^T A$  is singular or nearly so, adding a multiple of the identity repairs it:

$$(A^T A + \delta^2 I)\hat{\mathbf{x}} = A^T \mathbf{b}, \quad \delta > 0.$$

The matrix on the left is positive definite for every  $\delta > 0$ , since  $\mathbf{x}^T(A^T A + \delta^2 I)\mathbf{x} = \|A\mathbf{x}\|^2 + \delta^2\|\mathbf{x}\|^2$ . In terms of the singular value decomposition each  $\sigma_i$  is replaced by  $\sigma_i/(\sigma_i^2 + \delta^2)$ , so small singular values are damped rather than inverted.

**Remark 9.4.1** Statistics calls this ridge regression and deep learning calls it weight decay, but the linear algebra is identical: trade a little bias for a large reduction in sensitivity to noise. As  $\delta \rightarrow 0$  the solution approaches the pseudoinverse answer of Section 7.5.

# Linear Algebra behind Learning from Data

# 10

Sources: Strang, Introduction to Linear Algebra, 6th ed. [8], Chapter 10; Zhang, Lipton, Li and Smola, Dive into Deep Learning [10], Chapter 2.

This chapter closes Part I by putting the factorisations to work on data. Nothing new is proved; the point is to see that the objects of the previous nine chapters are the objects a learning system manipulates.

|                                                   |    |
|---------------------------------------------------|----|
| 10.1 Data as a matrix . . . . .                   | 33 |
| 10.2 Centring and covariance . . . . .            | 33 |
| 10.3 Principal components . . . . .               | 34 |
| 10.4 Low rank as a modelling assumption . . . . . | 34 |
| 10.5 Batched computation . . . . .                | 34 |
| 10.6 What carries forward . . . . .               | 35 |

## 10.1 Data as a matrix

Store  $n$  examples with  $d$  features each as  $X \in \mathbb{R}^{n \times d}$ , one example per row. Two readings, both from Section 2.4, are used constantly:

**By rows** each row is a point in  $\mathbb{R}^d$ , and the dataset is a cloud of  $n$  such points.

**By columns** each column is one feature measured across all examples, and the column space of  $X$  is the set of quantities expressible as weighted mixtures of features.

A linear model is then  $\hat{y} = Xw$ , so the achievable predictions form exactly  $\text{Col}(X)$ , and fitting it by least squares is the projection of Chapter 4.

This is the sense in which linear algebra is not a prerequisite for learning but a description of it. The reachability question of Chapter 1 is the question of what a linear model can represent.

## 10.2 Centring and covariance

Subtracting the mean of each column gives the centred matrix  $X_0$ . The *sample covariance matrix* is

$$S = \frac{1}{n-1} X_0^T X_0 \in \mathbb{R}^{d \times d},$$

symmetric and positive semidefinite by Section 6.5, with  $s_{jj}$  the variance of feature  $j$  and  $s_{jk}$  the covariance between features  $j$  and  $k$ .

**Example 10.2.1** Let

$$X_0 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \\ -2 & -1 \\ -1 & -2 \end{bmatrix},$$

whose columns already sum to zero. Then  $X_0^T X_0 = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$  and  $S = \frac{1}{3} X_0^T X_0$ . The eigenvalues of  $X_0^T X_0$  are 18 and 2, with eigenvectors  $[1, 1]^T$  and  $[1, -1]^T$ .

## 10.3 Principal components

Principal component analysis asks for the direction  $\mathbf{q}$  of unit length along which the projected data vary most. That is the maximiser of  $\mathbf{q}^\top \mathbf{S} \mathbf{q}$ , and by the spectral theorem it is the eigenvector of  $S$  belonging to the largest eigenvalue.

**Proposition 10.3.1** *The principal directions of  $X_0$  are its right singular vectors, and the variance captured by the  $i$ th component is proportional to  $\sigma_i^2$ .*

*Proof.* Writing  $X_0 = U\Sigma V^\top$  gives  $X_0^\top X_0 = V\Sigma^\top \Sigma V^\top$ , which is the eigendecomposition of a symmetric matrix with eigenvalues  $\sigma_i^2$  and eigenvectors  $\mathbf{v}_i$ .  $\square$

**Example 10.3.1** (continued) Here  $\sigma_1 = \sqrt{18} = 3\sqrt{2}$  and  $\sigma_2 = \sqrt{2}$ , so the first component  $[1, 1]^\top / \sqrt{2}$  accounts for  $18/20 = 90\%$  of the total variance. Projecting onto it replaces two features by one while discarding a tenth of the spread.

Computing the singular value decomposition of  $X_0$  is preferable to forming  $S$  first, for the conditioning reason given in Section 9.2.

## 10.4 Low rank as a modelling assumption

Eckart–Young (Section 7.4) says the truncated sum  $A_k = \sum_{i \leq k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$  is the best rank- $k$  approximation. Three familiar techniques are that theorem applied to different matrices: dimensionality reduction applies it to a data matrix, compression of a trained layer applies it to a weight matrix, and latent-factor recommendation applies it to a partially observed ratings matrix.

**Remark 10.4.1** The assumption being made each time is that the interesting structure lives in few directions and the rest is noise. Whether that is true is a modelling question about the data, not a mathematical one — the theorem is silent about where to cut.

## 10.5 Batched computation

A linear layer applied to a batch is one matrix product. With  $X \in \mathbb{R}^{n \times d}$  and weights  $W \in \mathbb{R}^{d \times h}$ ,

$$Z = XW + \mathbf{1}\mathbf{b}^\top \in \mathbb{R}^{n \times h},$$

where  $\mathbf{1} \in \mathbb{R}^n$  broadcasts the bias across rows. The block reading of Section 2.4 is what makes this efficient: the product decomposes into independent blocks that hardware evaluates in parallel, which is the practical reason mini-batches exist at all.

**Remark 10.5.1** Bias makes the layer *affine* rather than linear, so it violates  $T(\mathbf{0}) = \mathbf{0}$  from Section 8.1. Nothing breaks — an affine map is a linear map followed by a shift — but statements proved for linear

maps have to be applied to the linear part alone.

## 10.6 What carries forward

Four ideas from Part I are used without further comment in the rest of the book: the column space as the set of reachable outputs; orthogonal projection as the solution of a least-squares problem; the eigenvalues of a symmetric matrix as the curvature of a quadratic; and the singular value decomposition as the canonical way to separate signal from remainder. Chapter 22 returns to them once the calculus and probability are in place.



# **CALCULUS FOR LEARNING**

Sources: Hass, Heil and Weir; Thomas' Calculus, 14th ed. [2], Chapters 2–3; MIT 18.01 [4].

Part II covers only the calculus that optimisation needs. That means differentiation, and in particular the chain rule, which is the entire mathematical content of training a network by gradients. Integration appears only where Chapter 17 needs it.

|                                                           |    |
|-----------------------------------------------------------|----|
| 11.1 The derivative as a local linear approximation . . . | 37 |
| 11.2 The rules . . . . .                                  | 37 |
| 11.3 The chain rule . . . . .                             | 38 |
| 11.4 Long compositions . . . .                            | 38 |
| 11.5 What is not covered . . .                            | 38 |

## 11.1 The derivative as a local linear approximation

**Definition 11.1.1** For  $f : \mathbb{R} \rightarrow \mathbb{R}$ , the derivative at  $a$  is

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h},$$

when the limit exists.

The definition is a statement about approximation. Rearranged, it says

$$f(a+h) = f(a) + f'(a)h + o(h),$$

where the remainder shrinks faster than  $h$ . In words: near  $a$ , the function is the straight line  $f(a) + f'(a)h$ , plus an error too small to matter at first order.

Differentiability implies continuity, but not conversely:  $f(x) = |x|$  is continuous at 0 and has no derivative there, because the one-sided limits disagree. This matters in practice, since the rectified linear unit  $\max(0, x)$  has exactly this kink and is handled by convention rather than by theorem.

This reading is the one that generalises. In Chapter 12 the straight line becomes a plane and  $f'(a)$  becomes a vector; in Chapter 13 the plane becomes an affine map and the vector becomes a matrix. The idea never changes.

## 11.2 The rules

For differentiable  $f$  and  $g$ ,

$$(f+g)' = f' + g', \quad (fg)' = f'g + fg', \quad \left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2},$$

together with  $(x^n)' = nx^{n-1}$ ,  $(e^x)' = e^x$ , and  $(\ln x)' = 1/x$  for  $x > 0$ .

**Example 11.2.1** The logistic function  $\sigma(z) = \frac{1}{1+e^{-z}}$  satisfies

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)(1-\sigma(z)).$$

The derivative is expressible in terms of the value itself, which is why

implementations cache  $\sigma(z)$  during the forward pass and reuse it going backwards.

### 11.3 The chain rule

**Theorem 11.3.1** *If  $g$  is differentiable at  $a$  and  $f$  is differentiable at  $g(a)$ , then  $f \circ g$  is differentiable at  $a$  and*

$$(f \circ g)'(a) = f'(g(a)) g'(a).$$

The linear-approximation reading makes the statement almost obvious:  $g$  multiplies a small input change by  $g'(a)$ , then  $f$  multiplies the result by  $f'(g(a))$ , so the composed effect is the product of the two factors.

**Example 11.3.1** For  $f(x) = \sigma(wx + b)$  with  $w, b$  fixed,  $f'(x) = \sigma'(wx + b) \cdot w$ . Differentiating instead with respect to the *parameter*  $w$  gives  $\sigma'(wx + b) \cdot x$ . Same rule, different variable held fixed — and it is the second version that training uses.

Almost every gradient in a neural network is this calculation repeated. What makes networks distinctive is not the rule but the number of times it is applied and the care taken over the order of the arithmetic.

### 11.4 Long compositions

A network of  $L$  layers is a composition  $f = f_L \circ f_{L-1} \circ \cdots \circ f_1$ , and the chain rule gives

$$f'(x) = f'_L(z_{L-1}) \cdot f'_{L-1}(z_{L-2}) \cdots f'_1(x),$$

where  $z_k$  is the output of layer  $k$ . Two consequences deserve naming even in the scalar case.

First, the derivative is a *product* of  $L$  factors. If the factors are systematically smaller than one the product decays geometrically; if systematically larger it explodes. This is the vanishing and exploding gradient problem, visible here with no matrices involved.

Second, the intermediate values  $z_k$  are needed to evaluate the factors. They are computed on the way forward and must be kept, which is why memory during training scales with depth.

**Remark 11.4.1** Multiplication is associative but not equally cheap in every order. Accumulating the product from the output end backwards keeps every intermediate quantity the same shape as the output; accumulating from the input end does not. Section 14.6 makes this precise once the objects are matrices.

### 11.5 What is not covered

Techniques of integration, infinite series, and differential equations are outside the scope agreed for these notes. Definite integrals appear in Chapter 17 only to the extent that expectations of continuous random variables require them, and the fundamental theorem of calculus is used there without a separate development.

Sources: Hass, Heil and Weir, Thomas' Calculus, 14th ed. [2], Chapter 14; MIT 18.02 [4].

A loss depends on many parameters at once, so the derivative has to become a vector. This chapter builds that vector and computes it for the handful of expressions that account for most of what appears in practice.

|                                                         |    |
|---------------------------------------------------------|----|
| 12.1 Partial derivatives . . . . .                      | 39 |
| 12.2 Direction of steepest ascent . . . . .             | 39 |
| 12.3 Gradients of the expressions that matter . . . . . | 40 |
| 12.4 A worked parameter gradient . . . . .              | 40 |
| 12.5 Convexity in one line . . . . .                    | 40 |

## 12.1 Partial derivatives

For  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ , the partial derivative  $\partial f / \partial x_i$  is the ordinary derivative with respect to  $x_i$  with the other coordinates held fixed. Nothing new is required to compute one; the novelty is in assembling them.

**Definition 12.1.1** *The gradient of  $f$  at  $\mathbf{x}$  is the column vector*

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_n \end{bmatrix}.$$

The first-order approximation of Section 11.1 becomes

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^T \mathbf{h} + o(\|\mathbf{h}\|), \quad (12.1)$$

so the gradient is the vector whose inner product with a displacement predicts the change in  $f$ .

## 12.2 Direction of steepest ascent

**Proposition 12.2.1** *Among unit vectors  $\mathbf{u}$ , the directional derivative*

$$\nabla f(\mathbf{x})^T \mathbf{u}$$

*is largest when  $\mathbf{u}$  points along  $\nabla f(\mathbf{x})$ , and its value there is  $\|\nabla f(\mathbf{x})\|$ .*

Hence  $-\nabla f$  is the direction of steepest descent. That single sentence is the justification for every method in Chapter 14.

*Proof.* By the Cauchy–Schwarz inequality,  $|\nabla f^T \mathbf{u}| \leq \|\nabla f\| \|\mathbf{u}\| = \|\nabla f\|$ , with equality exactly when  $\mathbf{u}$  is parallel to  $\nabla f$ .  $\square$

A related fact is that the gradient is perpendicular to the level set through  $\mathbf{x}$ : moving along a level set does not change  $f$ , so the inner product in Equation 12.1 must vanish for such displacements.

## 12.3 Gradients of the expressions that matter

Three formulas cover most cases encountered in this book. Throughout,  $\mathbf{a}$  is a fixed vector and  $A$  a fixed matrix.

### Proposition 12.3.1

$$\begin{aligned}\nabla(\mathbf{a}^\top \mathbf{x}) &= \mathbf{a}, \\ \nabla(\mathbf{x}^\top A \mathbf{x}) &= (A + A^\top) \mathbf{x}, \\ \nabla\left(\frac{1}{2} \|A \mathbf{x} - \mathbf{b}\|^2\right) &= A^\top (A \mathbf{x} - \mathbf{b}).\end{aligned}$$

*In particular the middle formula is  $2A\mathbf{x}$  when  $A$  is symmetric.*

*Third formula.* Write  $f(\mathbf{x}) = \frac{1}{2}(A\mathbf{x} - \mathbf{b})^\top (A\mathbf{x} - \mathbf{b})$  and expand at  $\mathbf{x} + \mathbf{h}$ :

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + (A\mathbf{x} - \mathbf{b})^\top A \mathbf{h} + \frac{1}{2} \|A \mathbf{h}\|^2.$$

The middle term is  $(A^\top (A\mathbf{x} - \mathbf{b}))^\top \mathbf{h}$  and the last is  $O(\|\mathbf{h}\|^2)$ ; comparing with Equation 12.1 identifies the gradient.  $\square$

**Remark 12.3.1** Setting the third gradient to zero gives  $A^\top A \mathbf{x} = A^\top \mathbf{b}$  — the normal equations of Equation 4.1. The geometric derivation in Chapter 4 and the calculus derivation here produce the same equations, as they must.

## 12.4 A worked parameter gradient

Take one logistic unit with squared loss on a single example  $(\mathbf{x}, y)$ :

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad \hat{y} = \sigma(z), \quad \ell = \frac{1}{2} (\hat{y} - y)^2.$$

Working outwards from the loss,

$$\frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y, \quad \frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}), \quad \frac{\partial z}{\partial \mathbf{w}} = \mathbf{x},$$

so that

$$\nabla_{\mathbf{w}} \ell = (\hat{y} - y) \hat{y}(1 - \hat{y}) \mathbf{x}, \quad \frac{\partial \ell}{\partial b} = (\hat{y} - y) \hat{y}(1 - \hat{y}).$$

Note the factor  $\hat{y}(1 - \hat{y})$ , which is near zero whenever the unit is confidently right or confidently wrong. Squared loss on a saturating unit therefore learns slowly from its worst mistakes — one of the reasons cross-entropy is preferred, as Chapter 19 explains.

## 12.5 Convexity in one line

A differentiable  $f$  is *convex* when

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) \quad \text{for all } \mathbf{x}, \mathbf{y},$$

that is, when the first-order approximation never overestimates. For convex  $f$  a point with  $\nabla f = \mathbf{0}$  is a global minimum, which is why convex problems are the ones that can be declared solved. The losses of interest

in Part IV are not convex in the parameters, and Chapter 14 says what survives of the theory anyway.

# 13

## Jacobians, Hessians, and Taylor Approximation

|                                              |    |
|----------------------------------------------|----|
| 13.1 The Jacobian . . . . .                  | 42 |
| 13.2 The chain rule in matrix form . . . . . | 42 |
| 13.3 The Hessian . . . . .                   | 43 |
| 13.4 Taylor approximation . . .              | 43 |
| 13.5 The Jacobian determinant                | 43 |

Sources: Hass, Heil and Weir, Thomas' Calculus, 14th ed. [2], Chapters 14–15; MIT 18.02 [4].

The gradient handles scalar-valued functions. Vector-valued functions need a matrix of derivatives, and second-order behaviour needs another. Both are already familiar objects from Part I wearing new labels.

Note the transposition. The gradient is a column by convention, the Jacobian of a scalar function a row. Keeping this straight is the source of most sign-free errors in gradient derivations.

### 13.1 The Jacobian

**Definition 13.1.1** For  $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ , the Jacobian at  $\mathbf{x}$  is  $J \in \mathbb{R}^{m \times n}$  with entries

$$J_{ij} = \frac{\partial f_i}{\partial x_j}.$$

Row  $i$  of  $J$  is the gradient of the  $i$ th output. The first-order approximation is

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + J\mathbf{h} + o(\|\mathbf{h}\|),$$

so the Jacobian is the matrix of the linear map that best imitates  $f$  near  $\mathbf{x}$ . When  $m = 1$  it is  $\nabla f^\top$ , a single row.

**Example 13.1.1** An affine layer  $f(\mathbf{x}) = W\mathbf{x} + \mathbf{b}$  has  $J = W$  everywhere: it is already linear, so its best linear approximation is itself. An elementwise activation  $f(\mathbf{x}) = (\phi(x_1), \dots, \phi(x_n))$  has  $J = \text{diag}(\phi'(x_1), \dots, \phi'(x_n))$ .

### 13.2 The chain rule in matrix form

**Theorem 13.2.1** If  $\mathbf{h} = \mathbf{g} \circ \mathbf{f}$  with  $f : \mathbb{R}^n \rightarrow \mathbb{R}^p$  and  $g : \mathbb{R}^p \rightarrow \mathbb{R}^m$ , then

$$J_{\mathbf{h}}(\mathbf{x}) = J_{\mathbf{g}}(f(\mathbf{x})) J_{\mathbf{f}}(\mathbf{x}).$$

The scalar chain rule of Section 11.3 is the case  $m = p = n = 1$ . The matrix version says a composition of maps corresponds to a product of Jacobians, in the same order as the composition — and since matrix multiplication is not commutative, the order is now load-bearing.

**Remark 13.2.1** A network is exactly such a composition, alternating affine maps and elementwise activations. Its Jacobian is therefore an alternating product  $W_L D_{L-1} W_{L-1} \cdots D_1 W_1$  with  $D_k$  diagonal. The vanishing and exploding behaviour anticipated in Section 11.4 is now a statement about the singular values of that product.

## 13.3 The Hessian

**Definition 13.3.1** For  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  with continuous second partials, the Hessian is  $H \in \mathbb{R}^{n \times n}$  with  $H_{ij} = \partial^2 f / \partial x_i \partial x_j$ .

Equality of mixed partials makes  $H$  symmetric, so the spectral theorem of Section 6.4 applies: it has real eigenvalues and an orthonormal eigenbasis. Those eigenvalues are the curvatures of  $f$  along the corresponding directions.

## 13.4 Taylor approximation

Extending Equation 12.1 one order further,

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^T \mathbf{h} + \frac{1}{2} \mathbf{h}^T H(\mathbf{x}) \mathbf{h} + o(\|\mathbf{h}\|^2). \quad (13.1)$$

The quadratic term is exactly the quadratic form of Section 9.3, so everything proved there transfers. In particular a positive definite Hessian makes the local model a bowl with a unique minimum.

**Proposition 13.4.1** (Second-derivative test) Let  $\nabla f(\mathbf{x}) = \mathbf{0}$ . If  $H(\mathbf{x})$  is positive definite the point is a local minimum; if negative definite, a local maximum; if it has eigenvalues of both signs, a saddle point.

**Example 13.4.1** For  $f(x_1, x_2) = x_1^2 + 3x_1x_2 + 2x_2^2$ ,  $H = \begin{bmatrix} 2 & 3 \\ 3 & 4 \end{bmatrix}$  with  $\det H = -1 < 0$ , so the eigenvalues have opposite signs and the origin is a saddle.

In high dimensions saddles vastly outnumber local minima, since a critical point is a minimum only if *every* one of  $n$  eigenvalues is positive. Much of the practical behaviour of gradient descent on deep models is about escaping saddles rather than escaping minima.

## 13.5 The Jacobian determinant

When  $m = n$ ,  $|\det J|$  measures how the map scales volume locally — the statement anticipated at the end of Chapter 5. It appears whenever variables are changed inside an integral:

$$\int_{g(\Omega)} f(\mathbf{y}) d\mathbf{y} = \int_{\Omega} f(g(\mathbf{x})) |\det J_g(\mathbf{x})| d\mathbf{x}.$$

**Remark 13.5.1** This is the one place Part II needs integration, and it is needed for Chapter 16: transforming a random variable transforms its density by exactly this factor. A map with  $\det J = 0$  collapses volume, which is the calculus reading of singularity.



# 14

## Gradient-Based Optimisation

|                                                        |    |
|--------------------------------------------------------|----|
| 14.1 Descent directions and the basic method . . . . . | 44 |
| 14.2 The quadratic case, exactly                       | 44 |
| 14.3 Momentum and adaptive steps . . . . .             | 45 |
| 14.4 Newton's method . . . . .                         | 45 |
| 14.5 Stochastic gradients . . . . .                    | 45 |
| 14.6 Backpropagation . . . . .                         | 45 |

Sources: Hass, Heil and Weir, Thomas' Calculus, 14th ed. [2], Chapter 14; Strang, Introduction to Linear Algebra, 6th ed. [8], Chapter 9; Zhang, Lipton, Li and Smola, Dive into Deep Learning [10], Chapters 11–12.

This chapter closes Part II by combining the two halves already assembled: the gradient says which way to move, and the Hessian says how far the move can be trusted.

### 14.1 Descent directions and the basic method

By Section 12.2,  $-\nabla f$  is the direction along which  $f$  falls fastest, so the simplest possible method is

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k), \quad (14.1)$$

with step size  $\alpha > 0$ . Everything interesting is in the choice of  $\alpha$ , and Equation 13.1 explains why: the linear model that justified the direction is only accurate while the quadratic term stays small.

### 14.2 The quadratic case, exactly

Take  $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$  with  $A$  symmetric positive definite, whose minimiser  $\mathbf{x}^*$  solves  $A\mathbf{x} = \mathbf{b}$  by Section 9.3. Since  $\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b} = A(\mathbf{x} - \mathbf{x}^*)$ , the error  $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}^*$  obeys

$$\mathbf{e}_{k+1} = (I - \alpha A) \mathbf{e}_k.$$

Diagonalising  $A = Q\Lambda Q^T$  decouples the coordinates: along the eigenvector  $\mathbf{q}_i$  the error is multiplied by  $1 - \alpha\lambda_i$  at every step.

**Theorem 14.2.1** *Gradient descent on this quadratic converges for every starting point if and only if  $0 < \alpha < 2/\lambda_{\max}$ . The best rate is obtained at  $\alpha^* = 2/(\lambda_{\min} + \lambda_{\max})$ , where the error contracts by a factor*

$$\frac{\kappa - 1}{\kappa + 1}, \quad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}},$$

*per step.*

With  $\kappa = 100$  the factor is 0.980; reducing the error by  $10^{-3}$  then takes about 340 steps. Conditioning, introduced in Section 9.2 as a numerical concern, is here a direct statement about training time.

**Remark 14.2.1** The geometry is the elongated ellipse of Section 9.3. The negative gradient points across the valley rather than along it, so the iterates zigzag. Nothing about this depends on the objective being artificial — an ill-conditioned Hessian produces the same behaviour for any smooth loss.

## 14.3 Momentum and adaptive steps

Momentum averages successive gradients,

$$\mathbf{v}_{k+1} = \beta \mathbf{v}_k + \nabla f(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \mathbf{v}_{k+1},$$

which cancels the alternating components of the zigzag while accumulating the consistent ones. Adaptive methods instead rescale each coordinate by a running measure of its own gradient magnitude, which amounts to approximating the diagonal of the Hessian on the fly.

**Remark 14.3.1** Both are attempts to reduce the effective condition number without forming second derivatives. Neither changes the fact, established above, that the condition number is what has to be reduced.

## 14.4 Newton's method

Minimising the quadratic model Equation 13.1 exactly, rather than following its linear part, gives

$$\mathbf{x}_{k+1} = \mathbf{x}_k - H(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k).$$

On a quadratic objective this reaches the minimum in a single step, and near a non-degenerate minimum it converges quadratically. The obstacle is cost: forming and factorising  $H$  is  $O(n^3)$  work in  $n$  parameters, which is unavailable when  $n$  is in the millions. Quasi-Newton methods maintain a cheap approximation instead.

## 14.5 Stochastic gradients

Training losses are sums over examples,  $f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \ell_i(\mathbf{x})$ . Evaluating the full gradient costs one pass over the data; evaluating it on a random mini-batch costs a fraction of that and gives an unbiased estimate, since the expectation of the batch gradient is the full gradient.

**Remark 14.5.1** The trade is variance for speed. Because the estimate is noisy, a constant step size leaves the iterates bouncing near the optimum rather than settling, which is why step sizes are decayed. The unbiasedness claim itself belongs to Chapter 17.

## 14.6 Backpropagation

For a composition  $f = f_L \circ \dots \circ f_1$  with a scalar output, the chain rule of Section 13.2 gives

$$\nabla f = J_1^T J_2^T \dots J_L^T,$$

a product that may be evaluated in either order. Multiplying from the left (inputs first) propagates full Jacobians, each of size (layer width) squared. Multiplying from the right (output first) propagates a single vector at every stage, because the output is scalar.

The name suggests a special algorithm. It is better understood as a choice of association order in a matrix product — the choice that keeps every intermediate object as small as the output rather than as large as the layer.

Backpropagation is that second ordering, nothing more: a forward pass storing the intermediate values, then a backward pass carrying one vector through the transposed Jacobians. Its cost is a small constant multiple of the forward pass, independent of the number of parameters, and that asymptotic fact is what makes training large models possible at all.

# **PROBABILITY FOR LEARNING**

# 15

## Sample Spaces, Conditioning, and Bayes

|                                                  |    |
|--------------------------------------------------|----|
| 15.1 Sample spaces and the axioms . . . . .      | 48 |
| 15.2 Conditional probability . . . . .           | 48 |
| 15.3 Total probability and Bayes' rule . . . . . | 48 |
| 15.4 Independence . . . . .                      | 49 |

Sources: Bertsekas and Tsitsiklis, Introduction to Probability, 2nd ed. [1], Chapter 1; MIT 6.041 [9], Lectures 1–4; MIT 18.05 [5].

Part III covers the probability that a learning problem needs: enough to state what a model assumes about its data, and enough to derive the two loss functions that dominate practice. It stops at estimation, as agreed.

### 15.1 Sample spaces and the axioms

An experiment is described by a *sample space*  $\Omega$ , the set of possible outcomes. An *event* is a subset of  $\Omega$ . A probability measure assigns to each event a number  $\Pr(A)$  subject to three requirements:

1.  $\Pr(A) \geq 0$ ;
2.  $\Pr(\Omega) = 1$ ;
3.  $\Pr(\bigcup_i A_i) = \sum_i \Pr(A_i)$  for disjoint events.

Everything else is bookkeeping on top of these three lines — including the results in Chapter 18 that justify learning from samples at all.

Immediate consequences are  $\Pr(A^c) = 1 - \Pr(A)$ , monotonicity, and the inclusion–exclusion formula  $\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B)$ .

### 15.2 Conditional probability

**Definition 15.2.1** For  $\Pr(B) > 0$ , the probability of  $A$  given  $B$  is

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}.$$

Conditioning is renormalisation:  $B$  becomes the new sample space, and the measure is rescaled so that it again totals one. For fixed  $B$  the map  $A \mapsto \Pr(A \mid B)$  satisfies the three axioms, so every result proved for probabilities holds for conditional probabilities as well.

Rearranging gives the multiplication rule  $\Pr(A \cap B) = \Pr(A \mid B)\Pr(B)$ , and chaining it,

$$\Pr(A_1 \cap \dots \cap A_n) = \Pr(A_1)\Pr(A_2 \mid A_1) \cdots \Pr(A_n \mid A_1 \cap \dots \cap A_{n-1}).$$

### 15.3 Total probability and Bayes' rule

Let  $B_1, \dots, B_k$  partition  $\Omega$ . Splitting any event along the partition gives the *total probability* formula

$$\Pr(A) = \sum_{i=1}^k \Pr(A \mid B_i)\Pr(B_i),$$

and dividing the multiplication rule by it gives *Bayes' rule*,

$$\Pr(B_j | A) = \frac{\Pr(A | B_j) \Pr(B_j)}{\sum_i \Pr(A | B_i) \Pr(B_i)}. \quad (15.1)$$

**Example 15.3.1** A test for a condition present in 0.1% of a population detects it in 99% of carriers and returns a false positive for 1% of non-carriers. For a positive result,

$$\Pr(\text{carrier} | +) = \frac{0.99 \times 0.001}{0.99 \times 0.001 + 0.01 \times 0.999} = \frac{0.00099}{0.01098} \approx 0.090.$$

A test that is right 99% of the time leaves the subject about 9% likely to be a carrier, because the prior is small enough that false positives outnumber true ones by roughly ten to one.

The same arithmetic explains why a classifier with excellent accuracy on a rare class can still produce mostly wrong positive predictions. Accuracy and precision answer different questions; Equation 15.1 is the conversion between them.

## 15.4 Independence

**Definition 15.4.1** Events  $A$  and  $B$  are independent when  $\Pr(A \cap B) = \Pr(A) \Pr(B)$ , equivalently  $\Pr(A | B) = \Pr(A)$  when  $\Pr(B) > 0$ .

Independence is an assumption about the mechanism generating the data, not a property that can be read off a formula. It is also fragile: events may be pairwise independent without being jointly independent, and independence generally fails once one conditions on a third event.

**Remark 15.4.1** The i.i.d. assumption of Chapter 20 is exactly independence applied to the examples in a dataset, plus the requirement that they share one distribution. Nearly every guarantee in Chapter 18 rests on it, and distribution shift is the name for what happens when the second half fails.

# 16

## Random Variables and Distributions

|                                                             |    |
|-------------------------------------------------------------|----|
| 16.1 Discrete random variables                              | 50 |
| 16.2 Continuous random variables . . . . .                  | 50 |
| 16.3 Joint, marginal, and conditional distributions . . . . | 51 |
| 16.4 Transforming a random variable . . . . .               | 51 |

Sources: Bertsekas and Tsitsiklis, Introduction to Probability, 2nd ed. [1], Chapters 2–3; MIT 6.041 [9], Lectures 5–12.

A random variable attaches a number to each outcome, which turns questions about events into questions about functions — and therefore into questions that calculus and linear algebra can answer.

### 16.1 Discrete random variables

**Definition 16.1.1** A random variable is a function  $X : \Omega \rightarrow \mathbb{R}$ . It is discrete when its range is finite or countable, and it is then described by its probability mass function  $p_X(x) = \Pr(X = x)$ , which is non-negative and sums to one.

Three that recur:

**Bernoulli**( $p$ )  $X \in \{0, 1\}$  with  $\Pr(X = 1) = p$ . The model for a single binary label.

**Binomial**( $n, p$ ) the number of successes in  $n$  independent Bernoulli trials,  $p_X(k) = \binom{n}{k} p^k (1-p)^{n-k}$ .

**Poisson**( $\lambda$ )  $p_X(k) = e^{-\lambda} \lambda^k / k!$ , the limit of Binomial( $n, \lambda/n$ ) as  $n \rightarrow \infty$ ; counts of rare events.

### 16.2 Continuous random variables

**Definition 16.2.1**  $X$  is continuous with density  $f_X \geq 0$  when

$$\Pr(a \leq X \leq b) = \int_a^b f_X(x) dx, \quad \int_{-\infty}^{\infty} f_X(x) dx = 1.$$

A density is not a probability:  $f_X(x)$  may exceed one, and  $\Pr(X = x) = 0$  for every single point. What has meaning is  $f_X(x) dx$ , the probability of a small interval.

The *cumulative distribution function*  $F_X(x) = \Pr(X \leq x)$  works for both kinds, is non-decreasing, and satisfies  $F'_X = f_X$  in the continuous case.

**Uniform**( $a, b$ ) constant density  $1/(b-a)$  on  $[a, b]$ .

**Exponential**( $\lambda$ )  $f_X(x) = \lambda e^{-\lambda x}$  for  $x \geq 0$ ; memoryless waiting time.

**Gaussian**( $\mu, \sigma^2$ )  $f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$ .

The Gaussian earns its place twice over: Chapter 18 explains why sums of many small effects look Gaussian, and Chapter 19 shows that assuming Gaussian noise turns maximum likelihood into least squares.

## 16.3 Joint, marginal, and conditional distributions

For two random variables the joint density  $f_{X,Y}$  determines everything. Integrating out one variable gives a *marginal*,

$$f_X(x) = \int f_{X,Y}(x, y) dy,$$

and dividing gives a *conditional*,  $f_{Y|X}(y | x) = f_{X,Y}(x, y)/f_X(x)$  wherever  $f_X(x) > 0$ . Independence is the statement  $f_{X,Y} = f_X f_Y$ .

**Remark 16.3.1** Supervised learning is the business of modelling  $f_{Y|X}$  and nothing else. The marginal  $f_X$  governs which inputs are seen, and a change in it without a change in  $f_{Y|X}$  is the mildest form of the distribution shift named in Chapter 21.

## 16.4 Transforming a random variable

If  $Y = g(X)$  with  $g$  differentiable and invertible, the density transforms by the Jacobian factor of Section 13.5:

$$f_Y(y) = f_X(g^{-1}(y)) |\det J_{g^{-1}}(y)|.$$

**Example 16.4.1** For  $X \sim \text{Gaussian}(0, 1)$  and  $Y = \sigma X + \mu$ , the inverse is  $(y - \mu)/\sigma$  with derivative  $1/\sigma$ , giving the  $\text{Gaussian}(\mu, \sigma^2)$  density. Standardising a feature is exactly this change of variables run backwards.

Densities need the Jacobian; expectations do not. Computing  $E[g(X)]$  requires no such correction, as Section 17.2 explains — a distinction worth keeping firmly separate.



# 17

## Expectation, Variance, and Covariance

|                                 |    |
|---------------------------------|----|
| 17.1 Expectation . . . . .      | 52 |
| 17.2 Expectations of functions  | 52 |
| 17.3 Variance . . . . .         | 52 |
| 17.4 Covariance and correlation | 53 |
| 17.5 The covariance matrix . .  | 53 |
| 17.6 Conditional expectation .  | 54 |

Sources: Bertsekas and Tsitsiklis, Introduction to Probability, 2nd ed. [1], Chapters 2–4; MIT 6.041 [9], Lectures 5–12.

Distributions are unwieldy objects. Expectation reduces one to a single number, variance to a second, and covariance extends the idea to pairs — producing, in the end, the covariance matrix already met in Chapter 10.

### 17.1 Expectation

**Definition 17.1.1** The expectation of  $X$  is

$$\mathbb{E}[X] = \sum_x x p_X(x) \quad \text{or} \quad \mathbb{E}[X] = \int x f_X(x) dx,$$

according as  $X$  is discrete or continuous, when the sum or integral converges absolutely.

**Theorem 17.1.1 (Linearity)** For random variables  $X, Y$  on the same space and constants  $a, b$ ,

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y].$$

No independence is required.

Linearity without independence is the workhorse of the subject. It is what makes the mini-batch gradient of Section 14.5 unbiased even though the examples in a batch share a model and a step.

### 17.2 Expectations of functions

The expectation of  $g(X)$  needs no new distribution:

$$\mathbb{E}[g(X)] = \sum_x g(x) p_X(x) \quad \text{or} \quad \mathbb{E}[g(X)] = \int g(x) f_X(x) dx.$$

Contrast Section 16.4: obtaining the *density* of  $g(X)$  requires a Jacobian factor, obtaining its *expectation* does not.

**Remark 17.2.1** Except for affine  $g$ ,  $\mathbb{E}[g(X)] \neq g(\mathbb{E}[X])$ . For convex  $g$  the inequality has a direction — Jensen’s inequality gives  $\mathbb{E}[g(X)] \geq g(\mathbb{E}[X])$  — which is why an average of losses is not the loss of an average.

### 17.3 Variance

**Definition 17.3.1**  $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$ , and the standard deviation

is its square root.

Expanding the square gives the computational form  $\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$ , and scaling behaves quadratically:  $\text{Var}(aX + b) = a^2 \text{Var}(X)$ . For independent  $X$  and  $Y$ ,  $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$ .

**Example 17.3.1** For  $X \sim \text{Bernoulli}(p)$ ,  $\mathbb{E}[X] = p$  and  $\mathbb{E}[X^2] = p$ , so  $\text{Var}(X) = p - p^2 = p(1 - p)$ , largest at  $p = \frac{1}{2}$  and zero at the extremes.

**Proposition 17.3.1** If  $X_1, \dots, X_n$  are i.i.d. with mean  $\mu$  and variance  $\sigma^2$ , the sample mean  $\bar{X}_n = \frac{1}{n} \sum_i X_i$  satisfies

$$\mathbb{E}[\bar{X}_n] = \mu, \quad \text{Var}(\bar{X}_n) = \frac{\sigma^2}{n}.$$

The  $1/n$  — and therefore the  $1/\sqrt{n}$  in the standard deviation — is the reason quadrupling a batch halves the gradient noise, and the reason the returns diminish so quickly.

## 17.4 Covariance and correlation

**Definition 17.4.1**  $\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$ , and the correlation is  $\rho = \text{Cov}(X, Y) / \sqrt{\text{Var}(X) \text{Var}(Y)} \in [-1, 1]$ .

Independence implies zero covariance; the converse is false. A standard counterexample is  $X$  symmetric about zero with  $Y = X^2$ : the two are completely dependent, yet  $\text{Cov}(X, Y) = \mathbb{E}[X^3] = 0$ .

**Remark 17.4.1** Covariance detects linear association only. Since it is exactly what principal component analysis in Section 10.3 consumes, that method sees linear structure and nothing else — a limitation of the tool, not of the data.

## 17.5 The covariance matrix

For a random vector  $\mathbf{X} \in \mathbb{R}^d$  with mean  $\boldsymbol{\mu}$ ,

$$\boldsymbol{\Sigma} = \mathbb{E}[(\mathbf{X} - \boldsymbol{\mu})(\mathbf{X} - \boldsymbol{\mu})^\top] \in \mathbb{R}^{d \times d}.$$

**Proposition 17.5.1**  $\boldsymbol{\Sigma}$  is symmetric positive semidefinite, and for any fixed  $\mathbf{a} \in \mathbb{R}^d$ ,

$$\text{Var}(\mathbf{a}^\top \mathbf{X}) = \mathbf{a}^\top \boldsymbol{\Sigma} \mathbf{a}.$$

*Proof.* Symmetry is immediate from the definition. For the second claim, expand  $\text{Var}(\mathbf{a}^\top \mathbf{X}) = \mathbb{E}[(\mathbf{a}^\top (\mathbf{X} - \boldsymbol{\mu}))^2] = \mathbf{a}^\top \mathbb{E}[(\mathbf{X} - \boldsymbol{\mu})(\mathbf{X} - \boldsymbol{\mu})^\top] \mathbf{a}$ . Since a variance is non-negative, the quadratic form is non-negative for every  $\mathbf{a}$ , which is positive semidefiniteness.  $\square$

So the population covariance matrix and the sample covariance matrix  $S$  of Section 10.2 are the same object, one exact and one estimated. The spectral theorem applies to both, which is what principal components rely on.

## 17.6 Conditional expectation

$\mathbb{E}[Y \mid X = x]$  is the mean of the conditional distribution, and  $\mathbb{E}[Y \mid X]$  is the random variable taking that value when  $X = x$ . The tower property states

$$\mathbb{E}[\mathbb{E}[Y \mid X]] = \mathbb{E}[Y].$$

**Proposition 17.6.1** *Among all functions  $g$ , the one minimising  $\mathbb{E}[(Y - g(X))^2]$  is  $g(X) = \mathbb{E}[Y \mid X]$ .*

**Remark 17.6.1** This identifies what a regression model trained on squared error is trying to be: the conditional mean of the target given the input. Everything the model gets wrong is either approximation error — the family cannot express that conditional mean — or estimation error from finite data.

Sources: Bertsekas and Tsitsiklis, Introduction to Probability, 2nd ed. [1], Chapter 5; MIT 6.041 [9], Lectures 19–20.

Two theorems justify the entire practice of learning from a sample: averages converge to what they estimate, and the error of the average has a predictable shape and size.

|                                             |    |
|---------------------------------------------|----|
| 18.1 Two inequalities . . . . .             | 55 |
| 18.2 The law of large numbers               | 55 |
| 18.3 The central limit theorem              | 56 |
| 18.4 What the theorems do not say . . . . . | 56 |

## 18.1 Two inequalities

**Proposition 18.1.1** (Markov) *If  $X \geq 0$  and  $a > 0$  then  $\Pr(X \geq a) \leq \mathbb{E}[X]/a$ .*

*Proof.*  $\mathbb{E}[X] \geq \mathbb{E}[X \cdot \mathbf{1}\{X \geq a\}] \geq a \Pr(X \geq a)$ , using non-negativity in the first step.  $\square$

**Proposition 18.1.2** (Chebyshev) *If  $\mathbb{E}[X] = \mu$  and  $\text{Var}(X) = \sigma^2$  then for  $a > 0$ ,*

$$\Pr(|X - \mu| \geq a) \leq \frac{\sigma^2}{a^2}.$$

*Proof.* Apply Markov to the non-negative variable  $(X - \mu)^2$  with threshold  $a^2$ .  $\square$

These bounds are crude — they use only two moments — but they hold for every distribution, which is exactly what is wanted when the distribution is unknown.

## 18.2 The law of large numbers

**Theorem 18.2.1** (Weak law) *Let  $X_1, X_2, \dots$  be i.i.d. with mean  $\mu$  and finite variance  $\sigma^2$ . Then for every  $\varepsilon > 0$ ,*

$$\Pr(|\bar{X}_n - \mu| \geq \varepsilon) \rightarrow 0 \quad \text{as } n \rightarrow \infty.$$

*Proof.* By Section 17.3,  $\text{Var}(\bar{X}_n) = \sigma^2/n$ . Chebyshev gives

$$\Pr(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\sigma^2}{n\varepsilon^2},$$

which tends to zero.  $\square$

**Remark 18.2.1** Training loss is an average over the sample, and test loss is an average over a different sample from the same distribution. The law is what allows either to stand in for the expected loss — and its reliance on the i.i.d. hypothesis is what makes distribution shift so damaging.

### 18.3 The central limit theorem

**Theorem 18.3.1** *With the same hypotheses and  $\sigma^2 > 0$ ,*

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \text{Gaussian}(0, 1).$$

The law of large numbers says the error vanishes; the central limit theorem says it vanishes at rate  $1/\sqrt{n}$  and that the rescaled error is Gaussian regardless of the distribution the samples came from.

This is the honest reason Gaussian noise is assumed so often. It is not that measurements are Gaussian by nature, but that an accumulation of many small independent contributions is approximately so.

**Example 18.3.1** Estimating an accuracy  $p$  from  $n$  independent test examples gives  $\text{Var}(\bar{X}_n) = p(1-p)/n$ . With  $p \approx 0.9$  and  $n = 1000$ , the standard deviation is about 0.0095, so two systems differing by half a percentage point on such a test set are not distinguishable by that test set alone.

### 18.4 What the theorems do not say

Both require independence and identical distribution, and the central limit theorem additionally requires finite variance. Heavy-tailed quantities can violate the last condition, and time-ordered data routinely violate the first. Where the hypotheses fail, the conclusions are not merely weaker — they can be false, and the confidence intervals built on them are then decorative.

Sources: Bertsekas and Tsitsiklis, Introduction to Probability, 2nd ed. [1], Chapters 8–9; MIT 6.041 [9], Lectures 21–25; MIT 18.05 [5].

Estimation is where probability meets the fitted model. This chapter shows that the two losses used almost universally in supervised learning are not arbitrary choices but maximum likelihood under two different noise assumptions — and that weight decay is a prior.

|                                 |    |
|---------------------------------|----|
| 19.1 The problem . . . . .      | 57 |
| 19.2 Maximum likelihood . . .   | 57 |
| 19.3 Two losses, one principle  | 58 |
| 19.4 Bayesian estimation . . .  | 58 |
| 19.5 What this part establishes | 59 |

## 19.1 The problem

Data  $x_1, \dots, x_n$  are assumed drawn from a family of distributions indexed by an unknown parameter  $\theta$ . An *estimator* is any function  $\hat{\theta}(x_1, \dots, x_n)$  of the data. It is a random variable, so it has a bias  $\mathbb{E}[\hat{\theta}] - \theta$  and a variance, and its quality is usually measured by mean squared error, which decomposes as

$$\mathbb{E}[(\hat{\theta} - \theta)^2] = (\mathbb{E}[\hat{\theta}] - \theta)^2 + \text{Var}(\hat{\theta}).$$

Bias and variance trade against each other. The regularisation of Section 9.4 buys a large reduction in the second term at the cost of a small increase in the first — which is why it works.

## 19.2 Maximum likelihood

**Definition 19.2.1** The likelihood is the joint density of the observed data read as a function of  $\theta$ ,

$$L(\theta) = \prod_{i=1}^n f(x_i | \theta),$$

and the maximum likelihood estimate is the  $\theta$  maximising it, equivalently maximising  $\log L(\theta) = \sum_i \log f(x_i | \theta)$ .

The logarithm is taken because it converts the product into a sum, which differentiates cleanly and does not underflow.

**Example 19.2.1** For  $X_i \sim \text{Bernoulli}(p)$ ,  $\log L(p) = \sum_i [x_i \log p + (1 - x_i) \log(1 - p)]$ . Differentiating and setting to zero gives  $\hat{p} = \bar{x}$ , the observed frequency.

**Example 19.2.2** For  $X_i \sim \text{Gaussian}(\mu, \sigma^2)$  with  $\sigma$  known,

$$\log L(\mu) = -\frac{1}{2\sigma^2} \sum_i (x_i - \mu)^2 + \text{const},$$

so  $\hat{\mu} = \bar{x}$ . Estimating  $\sigma^2$  as well gives  $\hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2$ , which is biased low; dividing by  $n - 1$  instead removes the bias.

## 19.3 Two losses, one principle

Now let the parameter be the weights of a model  $f_{\mathbf{w}}$  and condition on inputs.

**Gaussian noise gives squared error.** Assume  $y_i = f_{\mathbf{w}}(\mathbf{x}_i) + \varepsilon_i$  with  $\varepsilon_i \sim \text{Gaussian}(0, \sigma^2)$ , independent. Then

$$\log L(\mathbf{w}) = -\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2 + \text{const},$$

so maximising the likelihood is minimising the sum of squared errors. With a linear model this is precisely the least-squares problem of Chapter 4.

**Bernoulli labels give cross-entropy.** Assume  $y_i \in \{0, 1\}$  with  $\Pr(y_i = 1) = \hat{y}_i = f_{\mathbf{w}}(\mathbf{x}_i)$ . Then

$$-\log L(\mathbf{w}) = -\sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)],$$

which is the cross-entropy loss.

**Remark 19.3.1** A question left open in Section 12.4 is settled here. Squared error applied to a logistic output carries a factor  $\hat{y}(1 - \hat{y})$  that vanishes when the unit saturates; cross-entropy is the loss that the Bernoulli assumption actually implies, and its gradient with respect to the pre-activation reduces to  $\hat{y} - y$ , with no vanishing factor. The improvement is a consequence of matching the loss to the noise model rather than a numerical trick.

## 19.4 Bayesian estimation

Maximum likelihood treats  $\theta$  as fixed and unknown. The Bayesian alternative gives it a distribution: a *prior*  $p(\theta)$  before the data and a *posterior* after, related by Equation 15.1,

$$p(\theta \mid \text{data}) \propto L(\theta) p(\theta).$$

Reporting the maximiser of the posterior is *maximum a posteriori* estimation.

**Proposition 19.4.1** For a linear model with Gaussian noise of variance  $\sigma^2$  and a Gaussian prior  $\mathbf{w} \sim \text{Gaussian}(\mathbf{0}, \tau^2 I)$ , the maximum a posteriori estimate minimises

$$\frac{1}{2\sigma^2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \frac{1}{2\tau^2} \|\mathbf{w}\|^2,$$

which is ridge regression with  $\delta^2 = \sigma^2 / \tau^2$ .

*Proof.* Take logarithms in the display above. The log-likelihood contributes the first term up to a constant, and the log-prior of a Gaussian contributes the second. Maximising the sum is minimising its negative.  $\square$

## 19.5 What this part establishes

Three conclusions carry into Part IV. Averages over a sample estimate expectations, with error shrinking like  $1/\sqrt{n}$ . Squared error and cross-entropy are maximum likelihood under Gaussian and Bernoulli assumptions respectively. And regularisation is a prior, which means the choice of penalty is a statement of belief about the parameters rather than a free knob.

So the  $\delta^2 I$  added in Section 9.4 for numerical reasons, the weight decay added in training for empirical reasons, and a Gaussian prior on the weights are three descriptions of one modification.



# DEEP LEARNING FOUNDATIONS

Sources: Zhang, Lipton, Li and Smola, Dive into Deep Learning [10], Chapter 1.

Before any architecture is chosen, a learning problem has to be stated. This chapter fixes the vocabulary and the four moving parts that every supervised method rearranges, including the ones this project is preparing for.

## 20.1 Vocabulary

A few words are used with technical precision throughout Part IV.

**Machine learning** fitting a program's behaviour from data instead of specifying it by hand — *programming with data*.

**Deep learning** the subset in which the fitted function is a composition of many parameterised layers.

**Dataset** the collection of recorded observations.

**Model** the parameterised family of functions being fitted.

**Parameter** a number adjusted during training.

**Learning algorithm** the procedure that adjusts the parameters.

**Training** the act of running that procedure.

## 20.2 The four components

Any learning problem, however it is later dressed up, consists of four things that must be specified separately:

**Data** what the method is allowed to learn from;

**Model** the family of transformations available to it;

**Objective function** a number that says how badly the current model is doing;

**Algorithm** the procedure that adjusts the model to reduce that number.

### 20.2.1 Data

A dataset is a collection of *examples*, also called samples or data points. Each example is described by a fixed list of *features*, also called covariates, and in supervised problems it carries a *label* — the special attribute to be predicted, also called the target. The number of features is the *dimensionality* of the example, and when that number is the same for every example the data are of fixed length, which is what makes it possible to write a batch of  $n$  examples as a matrix  $X \in \mathbb{R}^{n \times d}$  and reuse everything from Chapter 10.

**Definition 20.2.1** *Examples are i.i.d. — independent and identically distributed — when each is drawn from the same distribution, independently of the others. Nearly every guarantee about generalisation assumes this,*

|        |                                              |    |
|--------|----------------------------------------------|----|
| 20.1   | Vocabulary . . . . .                         | 61 |
| 20.2   | The four components . .                      | 61 |
| 20.2.1 | Data . . . . .                               | 61 |
| 20.2.2 | Model . . . . .                              | 62 |
| 20.2.3 | Objective function . . .                     | 62 |
| 20.2.4 | Optimisation algorithm                       | 62 |
| 20.3   | Training, testing, and overfitting . . . . . | 62 |
| 20.4   | Why the framing is worth keeping . . . . .   | 63 |

Keeping “model” for the family and “parameters” for the numbers inside it avoids a persistent ambiguity: a trained model is a family *plus* a particular setting, and the two are chosen by different means.

Keeping the four apart is worth the pedantry. Most confusing results come from changing one of them silently — a new loss, a new sampling scheme — while attributing the difference to the model.

*and nearly every deployed system violates it to some degree.*

The formal content of that definition is Section 15.4, and the reason it matters is Section 18.2: without it, an average over the sample stops estimating anything in particular.

## 20.2.2 Model

The model is a parameterised map from features to predictions. Fixing the model family fixes what the method can express; fixing the parameters within that family fixes what it currently predicts. For a linear model the reachable predictions are exactly the column space of the data matrix, which is the representational question of Chapter 1 arriving in its applied form.

## 20.2.3 Objective function

The objective — the loss or cost function — turns the gap between prediction and label into a single number, lower being better. For a real-valued target the usual choice is squared error,

$$\ell(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2,$$

and for a categorical target it is cross-entropy. Section 19.3 shows that these two are not conventions but the maximum likelihood objectives for Gaussian and Bernoulli noise respectively.

**Remark 20.2.1** The quantity one actually cares about is often not differentiable — classification accuracy, ranking quality, revenue. What gets optimised is a tractable surrogate, and the gap between the surrogate and the real goal is a permanent source of surprise rather than a temporary inconvenience.

## 20.2.4 Optimisation algorithm

The algorithm searches the parameter space for a setting that makes the objective small. Gradient descent and its variants dominate, for the practical reason established in Section 14.6: the gradient of a composed differentiable model can be computed at roughly the cost of evaluating it, no matter how many parameters it has.

## 20.3 Training, testing, and overfitting

The available data are split. The *training set* is used to fit the parameters; the *test set* is held out and used only to estimate how the fitted model behaves on examples it has never seen.

The split exists because loss on the training set is a biased estimate of quality. A model flexible enough can drive training loss towards zero by memorising the particular examples it was given, including their noise. *Overfitting* is the name for the resulting gap: training loss keeps falling while test loss turns and rises.

A test set inspected repeatedly stops being a test set. Each look leaks information about it into the modelling decisions.

**Remark 20.3.1** Overfitting is not a defect of a specific model family. It is what happens whenever the capacity of the family is large relative to the information in the data, and it is the reason that model selection, regularisation, and honest held-out evaluation are part of the method rather than optional extras.

By Section 18.3, a test set of  $n$  examples estimates the true error with a standard deviation of order  $1/\sqrt{n}$ . Reported differences smaller than that are not evidence of anything, however carefully the experiment was run.

## 20.4 Why the framing is worth keeping

The four components are independent choices, and almost every method discussed in Chapter 21 is a different setting of them rather than a different subject. Recognising that keeps the number of things to learn small: new architectures change the model, new losses change the objective, and new optimisers change the algorithm, but the shape of the problem stays put.

# 21

## A Map of Learning Problems

|                                                   |    |
|---------------------------------------------------|----|
| 21.1 Supervised learning . . .                    | 64 |
| 21.1.1 Regression . . . . .                       | 64 |
| 21.1.2 Classification . . . . .                   | 64 |
| 21.1.3 Search and ranking . . .                   | 64 |
| 21.1.4 Recommender systems .                      | 65 |
| 21.1.5 Sequence learning . . .                    | 65 |
| 21.2 Unsupervised learning .                      | 65 |
| 21.3 Interacting with an<br>environment . . . . . | 65 |
| 21.4 Reinforcement learning                       | 66 |
| 21.5 Terminology carried<br>forward . . . . .     | 66 |

Sources: Zhang, Lipton, Li and Smola, Dive into Deep Learning [10], Chapter 1.

With the four components of Chapter 20 in place, the standard catalogue of problem types can be laid out as variations on them: what the data contain, what is being predicted, and whether the learner influences what it sees next.

### 21.1 Supervised learning

Every example carries a label, and the goal is to model the conditional distribution of the label given the features — the object  $f_{Y|X}$  of Section 16.3.

#### 21.1.1 Regression

The label is a real number and the natural loss is squared error, so by Section 17.6 the fitted function approximates  $\mathbb{E}[Y | X]$ . Everything in Chapter 4 applies verbatim when the model is linear.

#### 21.1.2 Classification

The label is one of finitely many *categories* or *classes*. Two classes make it binary classification; more make it multiclass. The output is a probability vector, and the loss is cross-entropy, which Section 19.3 derives rather than assumes.

*Hierarchical* classification exploits structure among the classes so that confusing two neighbouring categories is penalised less than confusing two distant ones. *Multi-label* classification drops the assumption that the categories are mutually exclusive, and models each label with its own Bernoulli output.

#### 21.1.3 Search and ranking

The output is an ordering of a candidate set rather than a single value, and quality depends on which items appear near the top. Losses here are built to approximate rank-based measures, which are themselves not differentiable — an instance of the surrogate problem noted in Subsection 20.2.3.

Accuracy is a poor summary when classes are unbalanced, for the reason computed in Section 15.3: with a rare positive class, most positive predictions can be wrong even at very high accuracy.

### 21.1.4 Recommender systems

Predict a user's response to an item from a sparse, partially observed matrix of past interactions. The classical treatment is a low-rank factorisation of that matrix, which is Section 10.4 applied to a table of ratings.

**Remark 21.1.1** Recommendation is the clearest case in which the fitted system changes the data it will later be trained on: what is recommended is what gets seen, and what gets seen is what gets recorded. Censoring, incentives, and feedback loops are open research problems, not solved engineering details.

### 21.1.5 Sequence learning

Input and output are sequences whose lengths need not match. Tagging and parsing attach structure to each position; automatic speech recognition maps audio to text; text-to-speech maps text to audio; machine translation maps a sequence in one language to another, generally of different length and different word order.

## 21.2 Unsupervised learning

No labels are supplied, and the goal is to describe the structure of the data itself.

**Clustering** group examples so that members of a group resemble each other more than they resemble members of other groups.

**Principal component analysis** find the few directions along which the data vary most — exactly Section 10.3.

**Causality and probabilistic graphical models** describe which variables influence which, rather than merely which co-vary.

**Generative adversarial networks** fit a mechanism that produces samples resembling the data, by training a generator against a discriminator.

Only the second of these is fully settled by linear algebra. Correlation is what a covariance matrix records, and Section 17.4 notes that it detects linear association only — causal structure is not recoverable from it without further assumptions.

### 21.3 Interacting with an environment

The problems above are *offline*: a fixed dataset is collected first, the model is fitted second, and the model never influences what it is shown. This is convenient and often unrealistic.

**Definition 21.3.1** Distribution shift occurs when the distribution generating the data at deployment differs from the one that generated the training data.

Since Section 18.2 is what licensed the transfer from sample to population, and it assumes a single fixed distribution, shift is not a small perturbation of the theory — it removes the hypothesis the theory rests on.

## 21.4 Reinforcement learning

Here the learner is an *agent* placed in a loop with an environment. At each step it receives an *observation*, chooses an *action* according to its *policy*, and receives a *reward*; the action influences what is observed next.

Two difficulties distinguish this from supervised learning. The reward may arrive long after the action responsible for it, so the agent faces the *credit assignment* problem. And the agent must balance exploiting what it already believes to be good against exploring alternatives it has not tried.

**Markov decision process** the standard model, in which the next state depends only on the present state and the chosen action.

**Multi-armed bandit** the one-state case, in which the agent chooses repeatedly among fixed options with unknown rewards.

**Contextual bandit** a bandit in which an observation is supplied before each choice, but actions still do not affect future states.

The bandit cases are where the estimation machinery of Chapter 19 applies most directly: each arm's value is a mean to be estimated, and the exploration problem is about how confident those estimates are.

## 21.5 Terminology carried forward

A last group of words appears throughout the literature and is worth fixing before Part V: *estimation*; *neural network*; *layer*; *kernel method*; *decision tree*; *graphical model*; and *representation learning* — the idea that the useful features should be learned along with the predictor rather than designed in advance. That last term is the one Chapter 22 takes up.

**WHERE THE HALVES MEET**



# 22 From Column Space to Linear Layers

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 22.1 A network is a composition of the objects already met . . . . . | 68 |
| 22.2 Three questions, three answers already given . .                | 68 |
| 22.3 Where the linear picture runs out . . . . .                     | 69 |
| 22.4 What to read next . . . . .                                     | 69 |

Sources: *Strang*, Introduction to Linear Algebra, 6th ed. [8], Chapter 10; *Zhang, Lipton, Li and Smola*, Dive into Deep Learning [10], Chapters 1–5.

The four parts of this book were assembled separately. This closing chapter says how they meet, and states — deliberately conservatively — what remains to be studied before the research question that gives the book its title can be approached.

## 22.1 A network is a composition of the objects already met

A feed-forward network alternates two kinds of map. The affine part,

$$Z = XW + \mathbf{1b}^T,$$

is the batched matrix product of Section 10.5; the activation part applies a fixed scalar function elementwise. Nothing in either is new. What the composition adds is depth, and depth is what makes the resulting function non-linear.

**Remark 22.1.1** Stacking affine maps with no activation between them achieves nothing. A product of matrices is a matrix, so the composition collapses to one linear map with the same column space. The activation is the only reason a second layer is worth having, which is a statement about Section 2.4 rather than about biology.

## 22.2 Three questions, three answers already given

**What can the model represent?** For a single linear layer the reachable outputs are exactly  $\text{Col}(W^T)$  applied to the inputs, so the rank of the weight matrix bounds what the layer can express — the reachability question of Subsection 1.3.1 in applied clothing. Compressing a trained layer by truncating its singular values, as in Section 10.4, is a deliberate reduction of that rank.

**How is it fitted?** By gradient descent, whose direction is justified by Section 12.2, whose step size is constrained by the curvature in Equation 13.1, whose convergence rate is governed by the condition number in Section 14.2, and whose cost is made acceptable by the association order in Section 14.6.

**Why does the fitted model generalise?** Because training loss is a sample average of an expectation (Section 18.2), because the deviation of that average shrinks like  $1/\sqrt{n}$  (Section 18.3), and because the loss itself is a likelihood, so that penalising the weights is placing a prior on them (Section 19.4).

## 22.3 Where the linear picture runs out

Every question above was answered by treating the layer as linear and the activation as an afterthought. That works because the analysis is local: the Jacobian of Section 13.2 is a genuine linear map, and the whole of Part I applies to it at each point. It does not describe the function globally, and it says nothing about which composition of layers should be preferred.

Two limitations are worth naming, because they are what motivates looking further:

**The non-linearity is fixed.** In a conventional layer the parameters are the entries of  $W$  and  $b$ ; the activation is chosen in advance and not learned. All the flexibility sits in the linear part.

**Interpretation is indirect.** The learned object is a large weight matrix whose singular vectors carry no meaning supplied by the problem. Understanding what a layer has learned is a separate research effort.

**Remark 22.3.1** Kolmogorov–Arnold networks are motivated by the first of these: they parameterise the univariate functions themselves rather than the linear mixing. Whether that reparameterisation delivers the advantages claimed for it is the question this repository exists to study, and it is not settled by anything in this book. **[needs verification]** — the technical content of that literature has not yet been read, and no claim about it is made here.

## 22.4 What to read next

The foundations assembled here are the standard ones, and they are sufficient to begin. The material that has not been covered, and that the next stage of these notes should add, is:

1. tensors and automatic differentiation in practice, continuing the notebooks already in this repository;
2. approximation theory for univariate functions, in particular spline bases, which is the mathematics a Kolmogorov–Arnold parameterisation rests on;
3. the Kolmogorov–Arnold representation theorem itself, in its original form, before any neural interpretation of it;
4. the primary literature on these networks, read against that background rather than as a substitute for it.

Part I through Part IV will be revised as that reading proceeds. The notes are intended to be maintained rather than finished.

The order matters. Reading the applied literature before the representation theorem invites accepting an analogy as a proof — which is the failure mode these notes are written to avoid.

# Bibliography

- [1] Dimitri P. Bertsekas and John N. Tsitsiklis. *Introduction to Probability*. Athena Scientific, Belmont, MA, second edition, 2008. ISBN 978-1-886529-23-6. Textbook for MIT 6.041.
- [2] Joel R. Hass, Christopher E. Heil, and Maurice D. Weir. *Thomas' Calculus*. Pearson, fourteenth edition, 2018. ISBN 978-0-13-443898-6.
- [3] Federico Marotta. kaobook: A  $\LaTeX$  Class for Books and Theses. <https://github.com/fmarotta/kaobook>.
- [4] 18.01 Single Variable Calculus and 18.02 Multivariable Calculus. MIT OpenCourseWare, Massachusetts Institute of Technology. <https://ocw.mit.edu/>.
- [5] Jeremy Orloff and Jonathan Bloom. 18.05 Introduction to Probability and Statistics. MIT OpenCourseWare, Massachusetts Institute of Technology, 2014. Course notes. <https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2014/>.
- [6] J. (Alex) Song. KaoNotes: A Research-Oriented  $\LaTeX$  Framework for Academic Notes. Version 0.1.1, 2026. DOI <https://doi.org/10.5281/zenodo.22018865>; concept DOI <https://doi.org/10.5281/zenodo.22018510>; repository <https://github.com/AlexandraSloan3/kaonotes>.
- [7] Gilbert Strang. 18.06 Linear Algebra. MIT OpenCourseWare, Massachusetts Institute of Technology, 2010. <https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/>.
- [8] Gilbert Strang. *Introduction to Linear Algebra*. Wellesley–Cambridge Press, Wellesley, MA, sixth edition, 2023. ISBN 978-1-7331466-7-8.
- [9] John N. Tsitsiklis. 6.041 Probabilistic Systems Analysis and Applied Probability. MIT OpenCourseWare, Massachusetts Institute of Technology, 2010. <https://ocw.mit.edu/courses/6-041-probabilistic-systems-analysis-and-applied-probability-fall-2010/>.
- [10] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. *Dive into Deep Learning*. Cambridge University Press, 2023. ISBN 978-1-009-38943-3. Online edition: <https://d2l.ai>.